

ToricGT III: Sheafified MCP Memory, Graph-Structured Vector Retrieval, and Long-Context Tropical Ring Attention

A NeurIPS-style third-iteration research article for model context protocols, markdown memory, knowledge graphs, 1–10M-token context windows, and bits-per-byte-safe reasoning agents

Amelie Schreiber
June 2026

Abstract

ToricGT I made toric geometry operational for graph-token reasoning by attaching finite BGG, Koszul, Gale-dual, and toric-chart certificates to hidden trajectories. ToricGT II replaced isolated certificates by a dynamical theory: a reasoning step became a standardized packet whose filtered simplicial complexes, multigraded persistence modules, sheaves, vector bundles, derived objects, and toric chart codes evolve under a learned graph-to-graph map. This paper develops ToricGT III. The object of study is no longer only a reasoning trajectory inside the model, but the external context substrate through which modern agents operate: model context protocols (MCPs), markdown-style memory, tool/resource/prompt traces, and knowledge graphs vectorized into embeddings with adjacency and provenance.

We define an MCP-memory object as a typed graph of messages, resources, tools, prompts, markdown blocks, knowledge-graph entities, schema fields, permissions, and update events. From it we build an efficient graph-structured vector database: every node, edge, simplex, sheaf section, Cox coordinate, and certificate summary receives a vector, a quantized code, adjacency links, and provenance metadata. A learned projection head maps the full database into a small retrieval space for approximate nearest-neighbor search, while graph expansion, sheaf gluing, toric quantization, and derived persistence rerank the candidates. The new long-context contribution is to treat the iterative reasoning context window itself as a finite but very large toric dynamical object: a 1–10M-token canvas that is progressively filled by reasoning blocks, MCP observations, markdown memories, knowledge-graph neighborhoods, verifier traces, and behavior-control sections until the required answer length $L_C \leq 10^7$ is reached. Tropical ring attention gives an exact max-plus streaming semantics for active evidence and provenance over the full context, while projected graph retrieval supplies a sublinear candidate generator. We prove stability, finite-generation, projection-preservation, quantization-error, tropical-ring exactness, stopping-time, sheaf-gluing, derived mapping-cone, and ergodic-average theorems that explain when such a system improves reasoning quality, retrieval accuracy, behavior control, and held-out bits-per-byte. Finally, we give a 40-objective training catalogue plus a long-context extension catalogue for 1–10M-token MCP/memory/KG reasoning.

Design contract. Toric geometry is used as a compression, retrieval, and control instrument. Cones, one-dimensional cones, affine semigroups, normal fans, Cox coordinates, sheaves, vector bundles, differential forms, Čech complexes, local cohomology, Stanley-Reisner rings, derived categories, tableaux, root-system combinatorics, and ergodic theory enter only through finite certificates, compact vector codes, low-rank probes, and auditable retrieval traces. Any structure that improves the teacher but harms quantized held-out bits-per-byte remains teacher-side. The 1–10M-token context window is governed by the same rule: the agent fills context only while marginal retrieved or generated slots improve net prediction, verification, or control.

Contents

1	Purpose, scope, and continuity with ToricGT I–II	2
1.1	Why a third paper is needed	2
1.2	Summary of ToricGT I	2
1.3	Summary of ToricGT II	2
1.4	What ToricGT III adds	3
2	MCPs as typed protocol graphs	3
2.1	Protocol primitives	3
2.2	Context records and prefix safety	4
3	Markdown memory and knowledge graphs	5
3.1	Markdown memory as a block poset	5
3.2	Knowledge graphs as typed quivers and path algebras	6
4	Filtered complexes and multigraded modules for context databases	6
4.1	From context graphs to complexes	6
4.2	Derived context objects and update maps	7
5	Graph-structured vector databases	8
5.1	Database object	8
5.2	Projected retrieval head	8
5.3	Training the projection	9
6	Toric quantization and Cox-coded context	10
6.1	Semigroup codes	10
7	Sheaves, vector bundles, and differential forms for retrieval control	11
7.1	Cellular sheaves over context graphs	11
7.2	Differential forms for evidence flow	11
8	Dynamics and ergodic theory of context databases	12
8.1	MCP updates as a dynamical system	12
8.2	Mixing, recurrence, and cache policy	12
9	Full-context iterative reasoning windows and tropical ring attention	13
9.1	The context window as a reasoning canvas	13
9.2	Context tokens, graph tokens, and MCP records	14
9.3	Tropical ring attention over a full context	14
9.4	Soft attention, tropical attention, and answer probability	16
9.5	Learned projected retrieval for a 10M-token context	16
9.6	Quantized toric codes for context windows	17
9.7	Filtered simplicial persistence of a growing context	18
9.8	Sheaves and global sections over a long window	18
9.9	Derived updates for MCP and memory edits	19
9.10	Ergodic theory of long-context fill dynamics	20
9.11	Stopping, sufficiency, and context length control	20
9.12	Implementation for a 1–10M-token MCP/memory/KG agent	21
9.13	Long-context training data construction	21

10 Long-context objective catalogue	22
10.1 LC1. Context Canvas Bits-per-Byte Utility	22
10.2 LC2. Tropical Ring Provenance Margin Loss	22
10.3 LC3. Projected Retrieval Gap Preservation	22
10.4 LC4. Quantized Score Robustness	23
10.5 LC5. Graph Expansion Recall Objective	23
10.6 LC6. MCP Capability Nonface Barrier	23
10.7 LC7. Markdown Cech Memory Gluing	23
10.8 LC8. KG Path Sheaf Consistency	23
10.9 LC9. Long-Context Occupation Entropy Regularizer	24
10.10LC10. Recurrence Utility Cache Loss	24
10.11LC11. Context Packing Submodularity Loss	24
10.12LC12. Reasoning Fill Monotonicity and Zigzag Alarm	24
10.13LC13. Derived Update Cone Loss	24
10.14LC14. Persistence Event Explanation Loss	25
10.15LC15. Tone/Personality Global Section Loss	25
10.16LC16. Evidence Boundary Accuracy Loss	25
10.17LC17. Tool-Call Schema Degree Loss	25
10.18LC18. Privacy and Permission Ideal Loss	25
10.19LC19. Tropical Segment-Tree Distillation	26
10.20LC20. Answer Stopping-Time Efficiency	26
11 Training objectives and losses	26
11.1 Base objective	26
11.2 Core retrieval objective	26
11.3 Forty objectives for ToricGT III	27
11.4 Part I: adapted ToricGT objectives	29
11.5 Part II: new MCP/memory/KG-native objectives	33
12 Worked examples and explicit computations	37
12.1 Example 1: a minimal MCP trace becomes a retrieval complex	37
12.2 Example 2: markdown-memory Cech residual	37
12.3 Example 3: path-algebra normal form in a knowledge graph	38
12.4 Example 4: a Cox context code with invalid coactivation	38
12.5 Example 5: retrieval projection margin calculation	38
12.6 Example 6: tableau compression of ontology schemas	38
13 Detailed architecture of the MCP retrieval head	39
13.1 Five-stage retrieval	39
13.2 Retriever as a morphism of sheafified graphs	39
13.3 Training data construction	39
14 Deployment blueprint for an MCP server	40
14.1 Server interface	40
14.2 Structured outputs	40
14.3 Memory write protocol	40
15 Additional theorem: context utility under perfect retrieval	41

16 Risk controls and failure modes **41**
16.1 False positives 41
16.2 Ablation ladder 41

17 Implementation methodology **42**
17.1 Offline build 42
17.2 Online retrieval 42
17.3 Training schedule 42

18 Evaluation protocol **43**

19 Discussion and limitations **43**

20 Conclusion **44**

A Additional proof details **44**
A.1 Submodular context packing 44
A.2 Graph expansion and recall 44

B Normalized record schema **44**

C Bibliography **45**

1 Purpose, scope, and continuity with ToricGT I–II

1.1 Why a third paper is needed

ToricGT I and II focus on the model’s internal reasoning geometry. That is necessary but insufficient for agents that reason by calling tools, reading resources, editing memories, and traversing knowledge graphs. In such agents, the context system is not a passive prompt prefix. It is an evolving mathematical object. A tool call creates a typed edge; a resource response creates a provenance-bearing section; a markdown memory edit changes a local chart; a knowledge-graph expansion changes adjacency; a retrieved context package changes future token probabilities. If those operations are treated as flat text concatenation, the model wastes capacity relearning locality, provenance, schema compatibility, and behavior constraints from surface tokens. The ToricGT III thesis is that these structures should be stored, retrieved, trained, and compressed as graph-structured algebraic objects.

The immediate engineering target is a vector database whose records are not independent vectors. Each record is a node or cell in a typed graph; its vector is indexed together with adjacency, filtration, sheaf restrictions, quantized toric chart code, permissions, freshness, provenance, and known failure modes. The retrieval head sees a projected version of this database, not the full expensive object. The model uses fast approximate search first, then graph expansion and exact finite audits second. This is the same design pattern used in the earlier papers: train with rich finite structure; deploy only the compressed pieces that improve held-out bits-per-byte, verified reasoning, or behavior control.

1.2 Summary of ToricGT I

ToricGT I represents structured tasks as typed graphs, tokenizes node and edge objects equivariantly, and uses graph-token Transformer blocks with ordinary attention, tropical attention, noncommutative-torus phase channels, GraphCG concept coordinates, Soft-MoE routing, GFlowNet graph-of-thought search, and trajectory memory. Its toric BGG layer adds finite certificates: sign-vector posets, standard filtrations, sparse differentials, Betti profiles, Gale-dual labels, active faces of Newton polytopes, toric-ideal relations, Euler-Koszul probes, and homological consistency checks. The central deployment rule is conservative: the deployable model retains a probe only if ablations show validation bits-per-byte lift, reasoning-quality lift, retrieval improvement, or behavior reliability after artifact-byte accounting and quantized export.

ToricGT I is most naturally viewed as a theory of *reasoning certificates*. It asks whether a hidden trajectory behaves like an exact finite algebraic object: a chain complex, a standard-object filtration, a toric module, a combinatorial resolution, or a behavior-corridor section. That framing already contains a memory component: trajectory retrieval should prefer helper paths with compatible BGG or resolution signatures, not merely nearby final embeddings.

1.3 Summary of ToricGT II

ToricGT II turns those certificates into a dynamical theory. A reasoning step is a padded packet

$$x_t = (H_t, G_t, M_t, V_t, B_t, C_t, \tau_t), \quad (1.1)$$

where H_t is a hidden table, G_t is a graph-of-thought neighborhood, M_t is memory state, V_t records verifier fields, B_t records behavior-corridor coordinates, C_t records finite certificates, and τ_t records a toric chart code. From x_t one constructs filtered complexes $\mathcal{K}_t$, multigraded persistence modules $\mathcal{M}_t$, derived objects $\mathcal{D}_t$, sheaf or vector-bundle charts $\mathcal{F}_t$, and toric embedding codes. The learned map $T_\theta : x_t \mapsto x_{t+1}$ induces dynamics on all of them. ToricGT II then proves finite-generation and stability statements, uses mapping-cone residuals to measure failures of functoriality, and applies ergodic averages to long-run bits-per-byte, topology, verifier accuracy, and behavior-corridor cost.

The key principle carried into ToricGT III is the side-information identity. If a prefix-visible context certificate Z_t helps predict the next byte B_t , then an optimal predictor can reduce expected log loss by

$$I(B_t; Z_t \mid \mathcal{F}_t) \quad (1.2)$$

bits, where $\mathcal{F}_t$ is the strict prefix sigma-field. If Z_t is decorative, future-dependent, too expensive, or not robust after quantization, it should not be serialized into the deployed system.

1.4 What ToricGT III adds

ToricGT III moves the mathematical object from *inside* a reasoning step to the context ecosystem around it. Its objects are MCP traces, markdown memories, graph-structured knowledge bases, vector indexes, retrieval heads, quantizers, and context-packing policies. The paper makes four contributions.

1. It defines a protocol-context graph category whose objects include MCP messages, tools, resources, prompts, results, markdown blocks, graph entities, edge relations, permissions, and provenance.
2. It turns markdown memory and knowledge graphs into sheaves, vector bundles, filtered complexes, and multigraded persistence modules, so that memory consistency and retrieval quality can be audited by finite algebra.
3. It constructs graph-structured vector databases with learned projections and quantized toric codes, and proves retrieval stability under projection and quantization margins.
4. It gives a trainable objective catalogue for improving bits-per-byte, reasoning quality, retrieval, accuracy, tone/personality control, safety boundaries, and artifact efficiency.

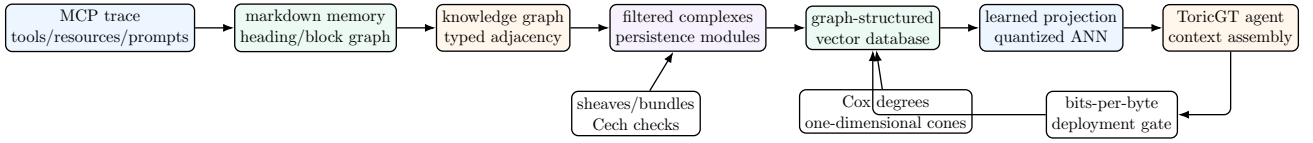


Figure 1: ToricGT III pipeline. MCP traces, markdown memory, and knowledge graphs are converted into filtered, sheafified, graph-structured vector databases. A learned projection enables efficient retrieval; toric and derived certificates audit the retrieved context before it affects the model.

2 MCPs as typed protocol graphs

2.1 Protocol primitives

The current MCP specification describes an open protocol for integrating LLM applications with external data sources and tools. It uses JSON-RPC 2.0 communication among hosts, clients, and servers; servers can expose resources, prompts, and tools, while clients can expose features such as sampling, roots, and elicitation. The tool feature is model-controlled: servers expose named tools with schemas, clients list tools, and tool calls return structured or unstructured content with error and security handling. ToricGT III abstracts these details into a finite typed graph so they can be embedded, indexed, audited, and trained.

Definition 2.1 (Protocol event alphabet). *Let T be a finite set of event types:*

$$\mathsf{T} = \{\text{message, tool, toolcall, toolresult, resource, prompt, schema, permission, memory, kg, verifier}\}. \quad (2.1)$$

A protocol event is a tuple

$$e = (\text{id}, \tau, t, \text{payload}, \text{schema}, \text{principal}, \text{provenance}, \text{visibility}), \quad (2.2)$$

where $\tau \in \mathbb{T}$, t is logical time, principal records the user, agent, server, or subsystem, and visibility records whether the event is prefix-visible for scoring.

Definition 2.2 (MCP context graph). *An MCP context graph is a typed attributed directed multigraph*

$$G_{\text{mcp}} = (V, E, s, t, \tau_V, \tau_E, x, e, p, \pi), \quad (2.3)$$

where vertices are protocol events, edges record temporal succession, call-return linkage, resource references, prompt instantiation, schema validation, permission flow, memory update, or knowledge-graph relation, x_v and e_a are bounded real and symbolic attributes, p is provenance metadata, and π is a prefix-visibility mask.

This graph representation deliberately separates semantic content from protocol legality. A tool call may be semantically useful but fail a schema constraint; a resource may be relevant but not visible under the current permission boundary; a memory block may be accurate but stale. These distinctions become graph features rather than prompt comments.

Proposition 2.3 (Finite traces give finite context graphs). *Every finite MCP interaction trace with finitely many messages, tool calls, resource references, prompts, and memory updates determines a finite MCP context graph. Relabeling internal event identifiers induces a graph isomorphism preserving types, temporal order, call-return edges, schemas, and provenance.*

Proof. The trace contains finitely many protocol events, so the vertex set is finite. Each declared relation among events is binary and belongs to a finite edge-type set; hence the edge multiset is finite. Replacing event identifiers by a bijection carries every event and every relation endpoint to the corresponding event and endpoint. Since types, schemas, time order, and provenance are stored as attributes rather than names, they are preserved. Thus identifier relabeling is graph isomorphism. $\square$

2.2 Context records and prefix safety

ToricGT III distinguishes three levels of context storage.

1. A *raw event store* contains exact JSON, text, files, and tool results.
2. A *structured graph store* contains typed vertices, edges, schemas, permissions, timestamps, and provenance.
3. A *projected vector index* contains compact codes used for fast retrieval.

The raw event store is authoritative for audit. The structured graph store is authoritative for legal and provenance constraints. The projected index is a speed layer and must be treated as lossy.

Definition 2.4 (Prefix-safe retrieval feature). *Let $\mathcal{F}_t$ be the sigma-field generated by the interaction prefix before byte b_t is scored. A retrieval feature Z_t is prefix-safe if Z_t is $\mathcal{F}_t$ -measurable. A training or evaluation run is prefix-safe if every retrieval, cache, graph edge, quantized code, and certificate used to predict b_t is prefix-safe.*

Theorem 2.5 (Side-information limit for context retrieval). *Let B_t be the next byte and let Z_t be a prefix-safe retrieval certificate. Among all normalized predictors, the expected log-loss improvement from revealing Z_t in addition to $\mathcal{F}_t$ is at most*

$$I(B_t; Z_t \mid \mathcal{F}_t) \quad (2.4)$$

bits, with equality for the true conditional distributions.

Proof. The optimal predictor without Z_t is $p(B_t | \mathcal{F}_t)$ and has expected base-two log loss $H(B_t | \mathcal{F}_t)$. The optimal predictor with Z_t is $p(B_t | \mathcal{F}_t, Z_t)$ and has expected loss $H(B_t | \mathcal{F}_t, Z_t)$. The difference is the conditional mutual information by definition:

$$H(B_t | \mathcal{F}_t) - H(B_t | \mathcal{F}_t, Z_t) = I(B_t; Z_t | \mathcal{F}_t). \quad (2.5)$$

Any other predictor has no lower expected log loss than the conditional distribution, by nonnegativity of conditional KL divergence. $\square$

This theorem is the compression discipline for the whole paper. A clever algebraic retrieval code is valuable only if it is prefix-visible and has conditional information about future bytes, verified answers, or behavior decisions. Otherwise it is a decorative source of latency and artifact bytes.

3 Markdown memory and knowledge graphs

3.1 Markdown memory as a block poset

Markdown memory has a useful latent geometry. Headings define a rooted block tree; links define cross-edges; bullets and tables define local product structure; code blocks define typed resources; front matter defines metadata. Treating a markdown file as flat text discards that geometry.

Definition 3.1 (Markdown block graph). *A markdown memory document D is parsed into a block graph*

$$G_D = (V_D, E_D), \quad (3.1)$$

where vertices are headings, paragraphs, bullets, table rows, code blocks, links, citations, and front-matter fields. There is a containment edge $u \rightarrow v$ when block v is inside block u , an order edge when v follows u inside the same parent, and a reference edge when a link, citation, or identifier connects two blocks. Each vertex has text c_v , metadata m_v , embedding z_v , and permission/freshness fields.

The block graph supports three important operations: restriction to a heading subtree, gluing of compatible local edits, and conflict detection across overlapping memories. These are sheaf operations.

Definition 3.2 (Memory sheaf). *Let P_D be the poset of markdown blocks ordered by containment, and give it the Alexandrov topology. A memory sheaf $\mathcal{F}_D$ assigns to each block v a vector space or finite set $\mathcal{F}_D(v)$ of admissible local sections: summaries, embeddings, provenance tuples, evidence claims, tone constraints, or tool-result schemas. For $u \preceq v$ there is a restriction map*

$$\rho_{vu} : \mathcal{F}_D(v) \rightarrow \mathcal{F}_D(u). \quad (3.2)$$

A global memory section is a compatible family (s_v) such that $\rho_{vu}(s_v) = s_u$ whenever $u \preceq v$.

Example 3.3 (A pragmatic memory conflict). *Suppose a project memory file has sections “dataset”, “license”, and “current experiment”. A retrieved tool result updates the dataset version. The license section restricts to a constraint “noncommercial”. A generated answer that combines the new dataset version with an old commercial-use statement is a Cech-gluing failure: two local sections agree on some lexical keys but disagree on a restriction induced by the license field.*

Definition 3.4 (Cech memory residual). *Let $\mathcal{U} = \{U_i\}$ be a finite cover of a memory block graph by heading neighborhoods or retrieval windows. If a model predicts local sections $s_i \in \mathcal{F}(U_i)$, define*

$$\mathcal{L}_C(s) = \sum_{i < j} \|\rho_{i,j}(s_i) - \rho_{j,i}(s_j)\|_{W_{ij}}^2 + \lambda \sum_{i < j < k} \|\delta s_{ijk}\|^2. \quad (3.3)$$

The second term penalizes higher-order inconsistencies on triple overlaps when available.

Proposition 3.5 (Zero residual glues local memory sections). *Assume $\mathcal{F}$ is a sheaf of vector spaces on a finite cover $\mathcal{U}$ and all restrictions are represented exactly. If $\mathcal{L}_{\tilde{C}}(s) = 0$ and the cover is closed under pairwise intersections, then the local sections s_i glue to a unique section on $\bigcup_i U_i$.*

Proof. The first term being zero says that s_i and s_j have equal restrictions on each overlap $U_i \cap U_j$. This is exactly the compatibility hypothesis in the sheaf gluing axiom. The sheaf axiom gives existence of a section s on the union with $s|_{U_i} = s_i$. If s' is another such section, then s and s' agree on every element of the cover, hence the locality axiom gives $s = s'$. $\square$

3.2 Knowledge graphs as typed quivers and path algebras

A knowledge graph is naturally a typed directed multigraph. Let $G = (V, E)$ with relation type map $\tau_E : E \rightarrow \mathcal{R}$. A path $p = e_k \cdots e_1$ has type word $\tau(p) = \tau(e_k) \cdots \tau(e_1)$. The path algebra $\mathbb{F}G$ is the vector space spanned by paths, with multiplication by concatenation when endpoints match and zero otherwise. This algebra encodes query composition and tool/resource chaining.

Definition 3.6 (Vectorized knowledge graph with adjacency). *A vectorized knowledge graph is a tuple*

$$\mathbb{K} = (G, Z, R, A, \Gamma), \quad (3.4)$$

where $G = (V, E)$ is a typed directed multigraph, $Z \in \mathbb{R}^{|V| \times d}$ are node embeddings, $R_\tau \in \mathbb{R}^{d \times d}$ are relation operators, $A_\tau \in \{0, 1\}^{|V| \times |V|}$ are typed adjacency matrices, and Γ stores provenance, permissions, freshness, and schema constraints.

A retrieval head should not only ask whether z_q is close to z_v . It should ask whether v lies in a graph neighborhood whose relations, schemas, and provenance make it usable for the current reasoning step.

Definition 3.7 (Path-consistent retrieval score). *For a query representation h_q , a candidate node v , and a path set $\mathcal{P}(q, v)$ from seed nodes to v , define*

$$S(q, v) = \langle P_q h_q, P_z z_v \rangle + \max_{p \in \mathcal{P}(q, v)} \sum_{e \in p} \omega_{\tau(e)} - \lambda_{\text{stale}} a_v - \lambda_{\text{perm}} \nu_v - \lambda_{\text{glue}} r_v. \quad (3.5)$$

Here P_q, P_z are learned projections, a_v is age or staleness, ν_v is permission risk, and r_v is a Cech or schema-gluing residual.

The score is intentionally decomposable. Its first term is fast vector search. The path term is graph expansion. The last terms enforce memory quality and trust.

4 Filtered complexes and multigraded modules for context databases

4.1 From context graphs to complexes

The vector database should expose more than nearest neighbors. A set of retrieved records has topology: multiple evidence nodes may jointly support a claim; a tool result may bridge two resources; a contradiction may appear as a forbidden coactivation; a style/tone constraint may be valid only with certain evidence and permission sections.

Definition 4.1 (Context filtration). *Let $\mathbb{D}$ be a graph-structured context database with node vectors z_v , edge types, trust scores t_v , freshness scores f_v , permission scores p_v , and role labels r_v . For a multifiltration index*

$$\alpha = (\rho, \theta_{\text{trust}}, \theta_{\text{fresh}}, \theta_{\text{perm}}, \theta_{\text{role}}) \in P, \quad (4.1)$$

define a simplicial complex $\mathcal{K}_\alpha(\mathbb{D})$ as follows. A vertex v is included if it passes the scalar thresholds. A simplex $\{v_0, \dots, v_k\}$ is included if all vertices are included, pairwise projected distances are at most ρ , and the induced typed subgraph belongs to an allowed motif set Ω .

Typical allowed motifs include: multiple resources supporting the same claim; tool-call/result/resource triples; markdown heading/child/link triples; knowledge-graph paths of bounded length; and behavior-corridor bundles containing evidence, verifier, tone, and answer-commitment nodes.

Definition 4.2 (Context persistence module). *For a field $\mathbb{F}$ and homological degree p , define*

$$\mathcal{M}_p(\mathbb{D})(\alpha) = H_p(\mathcal{K}_\alpha(\mathbb{D}); \mathbb{F}). \quad (4.2)$$

The structure maps are induced by inclusions when $\alpha \leq \beta$. If $P = \mathbb{N}^s$ or a finite grid in $\mathbb{N}^s$, this is a multigraded persistence module over $S = \mathbb{F}[u_1, \dots, u_s]$ after extending past the grid boundary.

Theorem 4.3 (Finite context persistence modules are finitely generated). *Let $P = \prod_{i=1}^s \{0, \dots, L_i\}$ be a finite grid and suppose the context database has n valid records. Extend $\mathcal{M}_p(\mathbb{D})$ constantly after the grid boundary. Then $\mathcal{M}_p(\mathbb{D})$ is a finitely generated multigraded module over $S = \mathbb{F}[u_1, \dots, u_s]$.*

Proof. There are finitely many grid points. At each grid point the complex has at most 2^n simplices, hence finite-dimensional homology. Choose a finite basis for every homology vector space at every grid point and place these basis elements as homogeneous generators in their grid degrees. The polynomial variables u_i propagate generators along coordinate directions. After the grid boundary, the module is constant, so no new generators are needed. Thus finitely many homogeneous generators suffice. $\square$

4.2 Derived context objects and update maps

The free resolution of a multigraded context persistence module is a compact algebraic summary of retrieval structure.

Definition 4.4 (Derived context object). *Let $\mathcal{M}_p(\mathbb{D})$ be a finite-grid context persistence module. A derived context object is a finite free resolution*

$$F_\bullet(\mathbb{D}) : 0 \rightarrow F_r \rightarrow \dots \rightarrow F_0 \rightarrow \mathcal{M}_p(\mathbb{D}) \rightarrow 0 \quad (4.3)$$

viewed up to quasi-isomorphism in $D^b(\text{gr-}S)$, enriched with role labels, Cox degrees, schema labels, permission levels, and provenance classes.

An MCP interaction updates the database. A new tool result may add a record; a memory edit may replace a section; a schema migration may rewrite fields; a knowledge-graph import may add many edges. This is a graph-to-graph map

$$U_\theta : \mathbb{D}_t \longrightarrow \mathbb{D}_{t+1}. \quad (4.4)$$

When U_θ is learned, we need to audit whether it induces valid maps on complexes and derived objects.

Definition 4.5 (Context chain-map residual). *Let $C_\bullet(\mathcal{K}_\alpha(\mathbb{D}_t))$ and $C_\bullet(\mathcal{K}_{s(\alpha)}(\mathbb{D}_{t+1}))$ be chain complexes for matched filtration indices. A proposed transport $F_{p,\alpha}$ has residual*

$$\text{Res}_\partial(F) = \sum_{p,\alpha} \|\partial_{t+1} F_{p,\alpha} - F_{p-1,\alpha} \partial_t\|_F^2. \quad (4.5)$$

Theorem 4.6 (Mapping cone measures memory-update failure). *Let $F_\bullet : C_\bullet \rightarrow D_\bullet$ be a degree-zero map between finite chain complexes. If $F_\bullet$ is a chain map, $\text{Cone}(F)$ is a complex and fits into a distinguished triangle $C_\bullet \rightarrow D_\bullet \rightarrow \text{Cone}(F) \rightarrow C_\bullet[1]$. If $F_\bullet$ is not a chain map, the obstruction to $\text{Cone}(F)$ being a complex is exactly $\partial_D F - F \partial_C$.*

Proof. The cone differential is $\delta(d, c) = (\partial_D d + Fc, -\partial_C c)$. Squaring gives

$$\delta^2(d, c) = (\partial_D^2 d + \partial_D Fc - F\partial_C c, \partial_C^2 c) = (\partial_D F - F\partial_C)c. \quad (4.6)$$

Thus $\delta^2 = 0$ precisely when F is a chain map. In that case the standard cone construction gives the distinguished triangle in the derived category. Otherwise the displayed commutator is the exact failure term. $\square$

5 Graph-structured vector databases

5.1 Database object

A standard vector database stores vectors and metadata. ToricGT III stores vectors as a graph-indexed sheafified complex.

Definition 5.1 (Graph-structured vector database). *A graph-structured vector database is a tuple*

$$\mathbb{D} = (G, Z, Y, Q, A, \mathcal{K}, \mathcal{F}, \Xi, \Pi), \quad (5.1)$$

where:

- $G = (V, E)$ is a finite typed attributed graph.
- $Z : V \rightarrow \mathbb{R}^d$ assigns a high-dimensional vector to each node.
- $Y : E \rightarrow \mathbb{R}^{d_e}$ assigns an edge vector or relation operator.
- $Q : V \cup E \rightarrow \mathcal{C}$ assigns quantized codes.
- A is the collection of typed adjacency matrices.
- $\mathcal{K}$ is a family of filtered complexes built from G, Z, Y, A .
- $\mathcal{F}$ is a cellular or Čech sheaf storing local sections and restrictions.
- Ξ stores exact finite certificates: Betti profiles, Čech residuals, schema checks, path-algebra normal forms, toric codes, and permission proofs.
- Π stores prefix-safety, freshness, security, and provenance policies.

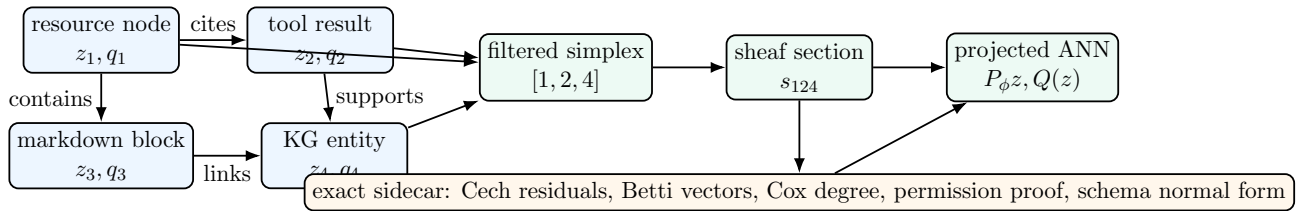


Figure 2: Graph-structured vector database. Vectors are attached to protocol, memory, and knowledge-graph nodes, but retrieval also uses adjacency, simplices, sheaf sections, quantized codes, and exact sidecar certificates.

5.2 Projected retrieval head

The database may store high-dimensional vectors and expensive certificates, but the first retrieval stage must be small. Let $P_\phi : \mathbb{R}^d \rightarrow \mathbb{R}^r$ be a learned projection with $r \ll d$.

Definition 5.2 (Projected graph retrieval). *Given a query packet x , produce a query vector h_x and a projected query $\tilde{h} = P_q h_x$. For each database node v , store $\tilde{z}_v = P_z z_v$ and a quantized code q_v . The stage-one candidate set is*

$$C_k(x) = \text{TopK}_{v \in V} \langle \tilde{h}, \tilde{z}_v \rangle - \lambda_q d_Q(q_x, q_v), \quad (5.2)$$

computed by an approximate nearest-neighbor index. The stage-two set is graph-expanded:

$$\overline{C}_k(x) = \{u : \text{dist}_G(u, C_k(x)) \leq R, \Pi(u, x) = \text{allow}\}. \quad (5.3)$$

The final context set is obtained by reranking $\overline{C}_k(x)$ with sheaf, schema, provenance, and certificate scores.

Theorem 5.3 (Projection-margin retrieval preservation). *Let the exact retrieval score for a finite candidate set C be s_v and the projected score be $\tilde{s}_v$. Suppose the true best candidate v^* has margin*

$$\gamma = s_{v^*} - \max_{v \neq v^*} s_v > 0. \quad (5.4)$$

If $|s_v - \tilde{s}_v| \leq \varepsilon$ for every $v \in C$ and $\gamma > 2\varepsilon$, then the projected top-1 candidate equals the exact top-1 candidate.

Proof. For every $v \neq v^*$,

$$\tilde{s}_{v^*} - \tilde{s}_v \geq (s_{v^*} - \varepsilon) - (s_v + \varepsilon) = s_{v^*} - s_v - 2\varepsilon \geq \gamma - 2\varepsilon > 0. \quad (5.5)$$

Thus v^* remains strictly highest-scoring under the projected scores. $\square$

Corollary 5.4 (Finite candidate Johnson-Lindenstrauss projection). *For a finite set of N query and database vectors with bounded norm, a random projection to dimension $r = O(\varepsilon^{-2} \log(N/\delta))$ preserves all pairwise inner products up to ε with probability at least $1 - \delta$ after normalization. Therefore top-1 and top- k retrieval are preserved whenever the exact score margins exceed the corresponding distortion bounds.*

Proof. The Johnson-Lindenstrauss lemma preserves pairwise distances among the N normalized vectors. Inner products are determined by squared distances through $\langle a, b \rangle = (\|a\|^2 + \|b\|^2 - \|a - b\|^2)/2$. Applying the previous theorem to the resulting inner-product distortion gives margin preservation. $\square$

5.3 Training the projection

A learned projection can be trained with three simultaneous constraints: retrieval accuracy, metric preservation on hard pairs, and quantization compatibility.

$$\mathcal{L}_{\text{retr}} = -\log \frac{\sum_{v \in P(x)} \exp S_\phi(x, v)/\tau}{\sum_{v \in P(x) \cup N(x)} \exp S_\phi(x, v)/\tau}, \quad (5.6)$$

$$\mathcal{L}_{\text{dist}} = \mathbb{E}_{(u,v) \in \mathcal{H}} (\langle Pz_u, Pz_v \rangle - \langle z_u, z_v \rangle)^2, \quad (5.7)$$

$$\mathcal{L}_{\text{qaware}} = \mathbb{E}_v \|Pz_v - \text{deq}(Q(Pz_v))\|^2. \quad (5.8)$$

Here $P(x)$ are positive records, $N(x)$ hard negatives, $\mathcal{H}$ hard pairs near decision boundaries, and Q is the quantizer. The final projection loss is

$$\mathcal{L}_{\text{proj}} = \mathcal{L}_{\text{retr}} + \lambda_d \mathcal{L}_{\text{dist}} + \lambda_q \mathcal{L}_{\text{qaware}} + \lambda_o \|P^\top P - I\|_F^2. \quad (5.9)$$

6 Toric quantization and Cox-coded context

6.1 Semigroup codes

Quantization is not only a numerical compression technique. It is also a combinatorial algebraic object. Let $M \simeq \mathbb{Z}^m$ be a lattice of context codes. Choose code atoms $A = \{a_1, \dots, a_s\} \subset M$, such as role, schema, source, tone, verifier state, trust band, and one-dimensional-cone chart identifiers. A record code is an element of the affine semigroup

$$S_A = \mathbb{N}A = \left\{ \sum_i n_i a_i : n_i \in \mathbb{N} \right\}. \quad (6.1)$$

The semigroup algebra $\mathbb{C}[S_A]$ records multiplicative composition of context features. Relations among codes produce a toric ideal:

$$I_A = \langle x^u - x^v : Au = Av \rangle. \quad (6.2)$$

A retrieval system should treat two code decompositions with the same semantic aggregate as equivalent unless provenance or permissions differ. Toric binomial losses enforce this equivalence.

Definition 6.1 (Cox context code). *Let Σ be a fan whose one-dimensional cones index context boundary types: evidence, verifier, memory, style, safety, schema, tool, resource, prompt, and answer commitment. A Cox context code is a monomial*

$$x^a = \prod_{\rho \in \Sigma(1)} x_\rho^{a_\rho} \quad (6.3)$$

with multidegree $\deg(x^a) \in \text{Cl}(X_\Sigma)$ or a learned finite quotient of that class group. Invalid coactivations are stored in an irrelevant or Stanley-Reisner ideal.

Theorem 6.2 (Quantized score perturbation bound). *Assume query and database vectors have norm at most R . Let $\hat{z}_v$ and $\hat{h}$ be quantized versions satisfying $\|z_v - \hat{z}_v\| \leq \eta$ and $\|h - \hat{h}\| \leq \eta$. Then*

$$|\langle h, z_v \rangle - \langle \hat{h}, \hat{z}_v \rangle| \leq 2R\eta + \eta^2. \quad (6.4)$$

Therefore a ranking margin greater than $2(2R\eta + \eta^2)$ is stable under quantization.

Proof. Write

$$\langle h, z \rangle - \langle \hat{h}, \hat{z} \rangle = \langle h - \hat{h}, z \rangle + \langle \hat{h}, z - \hat{z} \rangle. \quad (6.5)$$

Since $\|\hat{h}\| \leq R + \eta$, the absolute value is bounded by $\eta R + (R + \eta)\eta = 2R\eta + \eta^2$. The ranking statement follows by comparing the best candidate and any competitor, each of whose scores can shift by at most this amount. $\square$

Definition 6.3 (Toric quantization loss). *Let $c_v \in S_A$ be a discrete semigroup code and $\hat{c}_v$ a predicted code distribution. Let $\mathcal{B}$ be a set of low-degree toric binomial relations. Define*

$$\mathcal{L}_{\text{tq}} = \text{CE}(c_v, \hat{c}_v) + \lambda_{\text{bin}} \sum_{x^u - x^v \in \mathcal{B}} \left\| \sum_i u_i \ell_i - \sum_i v_i \ell_i \right\|^2 + \lambda_{\text{hole}} \text{Hole}(\hat{c}_v), \quad (6.6)$$

where ℓ_i are logit coordinates and Hole penalizes predicted codes outside the saturated semigroup or normalized codebook.

The practical benefit is compactness. Instead of storing long metadata strings for every record, the index stores small integer codes whose algebra is known. During retrieval, the model can combine codes by semigroup addition and reject invalid context packages by Stanley-Reisner nonface checks.

7 Sheaves, vector bundles, and differential forms for retrieval control

7.1 Cellular sheaves over context graphs

A cellular sheaf over a graph or simplicial complex attaches data to cells and restriction maps to incidences. In ToricGT III, a vertex section may be a local summary, an embedding, a permission token, a style state, or a schema object. An edge restriction describes how two records agree on a shared claim, entity, tool schema, or behavior corridor.

Definition 7.1 (Context sheaf on a complex). *Let K be a finite simplicial complex built from retrieved context records. A cellular context sheaf $\mathcal{F}$ assigns a vector space $\mathcal{F}(\sigma)$ to every simplex $\sigma \in K$ and a restriction map $\rho_{\tau\sigma} : \mathcal{F}(\tau) \rightarrow \mathcal{F}(\sigma)$ whenever $\sigma \preceq \tau$, satisfying identity and composition laws.*

Definition 7.2 (Retrieval vector bundle). *A retrieval vector bundle of rank r over a context complex K is a locally trivial sheaf of r -dimensional vector spaces. In implementation, every maximal retrieval window U_i has a local frame $B_i \in \mathbb{R}^{d \times r}$ and overlaps have transition matrices $g_{ij} \in GL_r$ satisfying the cocycle constraint*

$$g_{ij}g_{jk}g_{ki} = I \quad (7.1)$$

on triple overlaps.

The bundle is useful for tone/personality and behavior control. A user’s preferred technical register, a proof-rigor axis, a safe-refusal axis, and a citation-demand axis should be transported coherently across retrieved memories. If the transition maps drift, the answer may change style or confidence for reasons unrelated to evidence.

Definition 7.3 (Bundle-transition retrieval loss). *Given local frames B_i and predicted transitions g_{ij} , define*

$$\mathcal{L}_{\text{bundle}} = \sum_{i < j} \|B_i - B_j g_{ij}\|_F^2 + \lambda \sum_{i < j < k} \|g_{ij}g_{jk}g_{ki} - I\|_F^2. \quad (7.2)$$

7.2 Differential forms for evidence flow

Let the retrieved context graph have an oriented edge set. A discrete 1-form assigns a scalar or vector to each oriented edge. Evidence flow should often be conservative: if two paths derive the same claim from the same sources, their accumulated evidence should agree. Nonzero circulation flags contradiction, schema mismatch, or hidden dependence on tone rather than facts.

Definition 7.4 (Evidence 1-form). *Let $G = (V, E)$ be an oriented retrieved subgraph. An evidence 1-form is a function $\omega : E \rightarrow \mathbb{R}^r$ satisfying $\omega(v, u) = -\omega(u, v)$ when both orientations are present. Its discrete exterior derivative on a triangle (u, v, w) is*

$$(d\omega)_{uvw} = \omega_{uv} + \omega_{vw} + \omega_{wu}. \quad (7.3)$$

Definition 7.5 (Closed-form consistency loss). *For a retrieved complex K , define*

$$\mathcal{L}_{\text{form}} = \sum_{(u,v,w) \in K_2} \|\omega_{uv} + \omega_{vw} + \omega_{wu}\|^2 + \lambda \sum_{(u,v) \in E} \|\omega_{uv} - (\phi_v - \phi_u)\|^2, \quad (7.4)$$

where ϕ_v is an optional learned potential. The first term penalizes nonclosed evidence circulation; the second encourages exactness when path independence is expected.

8 Dynamics and ergodic theory of context databases

8.1 MCP updates as a dynamical system

Every interaction changes the context database. The update may add a tool result, refresh a resource, rewrite a markdown memory block, create a knowledge-graph edge, change permissions, or update quantized codes. Let $\mathfrak{D}$ be the state space of bounded graph-structured vector databases with fixed maximum nodes, maximum edges, finite codebooks, and bounded vectors. After padding, $\mathfrak{D}$ is compact.

Definition 8.1 (Context update kernel). *A ToricGT III context system has a Markov kernel*

$$P_\theta(\mathbb{D}, d\mathbb{D}') \quad (8.1)$$

that samples the next database after the agent observes a prefix, retrieves context, emits tokens, and processes tool or memory updates. Deterministic updates are the special case $\mathbb{D}' = U_\theta(\mathbb{D})$.

Theorem 8.2 (Finite quantized context chains admit invariant measures). *If all vectors and metadata are quantized into finite codebooks and the database has bounded capacity, then the induced context update process is a finite Markov chain. Hence it has at least one invariant probability measure.*

Proof. Bounded capacity and finite codebooks imply finitely many possible database states. A Markov kernel on a finite set is a stochastic matrix. Every finite stochastic matrix has a stationary distribution, for example by applying Brouwer’s fixed-point theorem to the probability simplex or by averaging iterates of any initial distribution and taking a convergent subsequence. $\square$

Theorem 8.3 (Ergodic averages for retrieval observables). *Let μ be an invariant probability measure for a context update map U_θ and let $\varphi \in L^1(\mu)$ be any observable: local bits-per-byte, retrieval precision, Cech residual, schema violation, permission risk, tone-corridor distance, quantization error, or Betti drift. Then*

$$\frac{1}{T} \sum_{t=0}^{T-1} \varphi(U_\theta^t \mathbb{D}) \quad (8.2)$$

converges for μ -almost every $\mathbb{D}$ to a U_θ -invariant function. If μ is ergodic, the limit is $\int \varphi d\mu$ almost surely.

Proof. This is Birkhoff’s ergodic theorem applied to the measure-preserving system $(\mathfrak{D}, \mu, U_\theta)$. $\square$

8.2 Mixing, recurrence, and cache policy

Ergodic quantities are not ornamental. They guide cache policy. If a context node is frequently recurrent and improves held-out bits-per-byte, it deserves a compact persistent code. If a node is visited often but causes verifier failures or style leakage, it should be demoted or isolated by permission/behavior nonfaces.

Define the empirical occupation measure over quantized toric cells $c \in \mathcal{C}$ by

$$\hat{\mu}_T(c) = \frac{1}{T} \sum_{t=0}^{T-1} \mathbf{1}\{Q(\mathbb{D}_t) = c\}. \quad (8.3)$$

The entropy rate of the symbolic context process is estimated by

$$\hat{h}_m = -\frac{1}{T-m} \sum_{t=m}^{T-1} \log \hat{p}(c_t \mid c_{t-m:t-1}). \quad (8.4)$$

A good memory system has enough entropy to adapt, but not so much that the same tasks trigger unrelated retrieval states.

9 Full-context iterative reasoning windows and tropical ring attention

9.1 The context window as a reasoning canvas

ToricGT III treats a long context window not as a flat buffer but as a finite canvas on which an iterative reasoning trajectory is written. The practical regime is large but finite: a model may have an admissible context capacity between one and ten million tokens, and a particular problem may require a realized context length

$$1 \leq L_C \leq C_{\max}, \quad C_{\max} = 10^7. \quad (9.1)$$

The object filled into this window is heterogeneous. Some positions are ordinary prompt or answer tokens; some are serialized MCP tool descriptors, tool calls, tool results, resources, prompts, roots, elicitation records, and sampling traces; some are markdown memory blocks; some are knowledge-graph neighborhoods; some are retrieved proofs, verifier reports, schema summaries, and behavior-corridor certificates. A long-context ToricGT agent should therefore reason over a typed graph-indexed canvas rather than a homogeneous token tape.

Definition 9.1 (Long-context reasoning canvas). *Fix a vocabulary $\mathcal{V}$, hidden dimension d , maximum context size $C_{\max}$, and a finite set of context types $\mathcal{T}$. A long-context reasoning canvas is a tuple*

$$\mathfrak{W} = (I, x, h, \tau, A, \mathcal{G}, \mathcal{K}, \mathcal{F}, \Xi), \quad (9.2)$$

where $I \subseteq \{1, \dots, C_{\max}\}$ is the set of filled positions, $x_i \in \mathcal{V} \cup \{\perp\}$ is a token or blank symbol, $h_i \in \mathbb{R}^d$ is a hidden state, $\tau_i \in \mathcal{T}$ is a type label, A is a typed adjacency relation on positions and database records, $\mathcal{G}$ is the induced typed context graph, $\mathcal{K}$ is a filtered simplicial complex built from graph cliques, typed hyperedges, or evidence coactivation, $\mathcal{F}$ is a sheaf of local memory/evidence sections over $\mathcal{K}$, and Ξ stores quantized toric chart codes, one-dimensional-cone ids, Cox degrees, permissions, and provenance. The realized length is $L(\mathfrak{W}) = |I|$.

The blank token $\perp$ is useful because a reasoning step can be standardized even when its text length varies. A complete thought step may occupy a block of b token positions, followed by padding to a fixed block size B_{step} . The padding is masked for language loss but remains useful for graph and sheaf alignment: the model can attach hidden summaries, verifier fields, persistence features, and retrieval pointers to a block even when the surface text is shorter.

Definition 9.2 (Reasoning fill trajectory). *A fill trajectory of horizon T is a sequence*

$$\mathfrak{W}_0 \subseteq \mathfrak{W}_1 \subseteq \dots \subseteq \mathfrak{W}_T, \quad (9.3)$$

where inclusion means that filled token positions, graph vertices, graph edges, simplices, local sections, and quantized chart metadata are extended rather than overwritten. At step t , a learned graph-to-graph-and-canvas map

$$F_\theta : \mathfrak{W}_t \times \mathbb{D}_t \times q \longrightarrow (\Delta \mathfrak{W}_t, \mathbb{D}_{t+1}, a_t) \quad (9.4)$$

chooses a reasoning block $\Delta \mathfrak{W}_t$, updates the external database $\mathbb{D}_t$, and emits either a continuation action or a terminal answer action a_t . The stopping time for answer production is

$$\tau_C = \inf\{t : a_t = \text{answer and } L(\mathfrak{W}_t) \leq C_{\max}\}. \quad (9.5)$$

This formulation is deliberately monotone for the main theory. Real systems revise memories and remove unsafe context. Those operations are modeled by a separate database update map $\mathbb{D}_t \mapsto \mathbb{D}_{t+1}$ and by zigzag persistence for the context complex. The realized canvas used for scoring remains prefix-causal: once a byte has been scored, later memory rewrites cannot be used to rescore it.

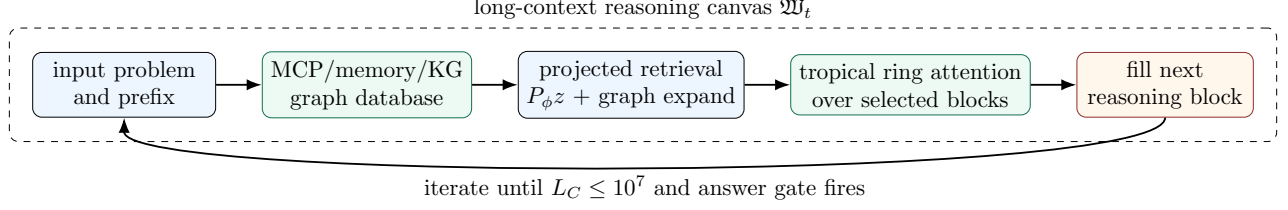


Figure 3: A 1–10M-token context is filled by repeated retrieval, tropical ring aggregation, and graph-structured reasoning blocks. The ring attention path gives exact max-plus provenance over selected or streamed blocks, while the projected retrieval path keeps search sublinear.

9.2 Context tokens, graph tokens, and MCP records

Let $\mathbb{D}$ be the external context database. It contains three interacting layers:

1. a raw layer of byte strings, markdown files, resource payloads, tool schemas, prompts, call/response traces, and knowledge-graph triples;
2. a graph layer $G_{\mathbb{D}} = (V, E)$ whose vertices are typed records and whose edges are containment, citation, temporal-successor, schema-field, capability, provenance, permission, entity-relation, and analogy links;
3. a vector/certificate layer with embeddings $z_v \in \mathbb{R}^d$, projected vectors $u_v = P_\phi z_v \in \mathbb{R}^r$, quantized codes q_v , sheaf sections s_v , filtered-complex memberships, toric chart ids, and small finite certificate summaries.

The canvas $\mathfrak{M}_t$ is a finite excerpt of this database together with generated reasoning text. The excerpt may be large, but it is not arbitrary: it is produced by graph retrieval, permissions, context packing, and a final bits-per-byte gate.

Definition 9.3 (Context incidence map). *A context incidence map is a function*

$$\iota_t : I_t \longrightarrow V(G_{\mathbb{D}_t}) \cup V_{\text{gen},t} \quad (9.6)$$

from filled token positions to either database records or generated reasoning vertices. The map is compatible with adjacency when a token span from a record induces a connected interval in the context graph and when generated reasoning steps attach edges to the records they cite, transform, summarize, or verify.

The incidence map is important for evaluation. If an answer cites a tool result, the answer tokens and the result tokens should have a short path in $G_{\mathbb{D}_t}$ and a compatible sheaf restriction. If a generated paragraph changes tone because of a style memory, the paragraph should be adjacent to that memory in the behavior subgraph. If the path is absent, the model may be using ungrounded latent memory rather than the retrieved context.

9.3 Tropical ring attention over a full context

Quadratic attention over 10^7 tokens is not the intended primitive. ToricGT III uses a mixture of projected retrieval, sparse graph attention, and tropical ring attention. The tropical component is exact for max-plus aggregation and therefore ideal for provenance, active evidence, shortest-path-like reasoning, dynamic-programming recurrences, and hard evidence maxima.

Let $Q \in \mathbb{R}^{m \times d}$ be a small query table, $K, V \in \mathbb{R}^{L_C \times d}$ be key and value tables for a selected or streamed context, and let $\mathcal{B} = \{B_1, \dots, B_R\}$ be a partition of the context positions into blocks. For

query i and output channel c , define

$$Y_{ic} = \max_{1 \leq j \leq L_C} \{\langle Q_i, K_j \rangle + V_{jc} + M_{ij}\}, \quad (9.7)$$

where $M_{ij} \in \{0, -\infty\}$ is a prefix, permission, type, or graph-sparsity mask.

Definition 9.4 (Tropical ring summary). *The block summary for block B_r is*

$$Y_{ic}^{(r)} = \max_{j \in B_r} \{\langle Q_i, K_j \rangle + V_{jc} + M_{ij}\}, \quad (9.8)$$

with provenance set

$$A_{ic}^{(r)} = \arg \max_{j \in B_r} \{\langle Q_i, K_j \rangle + V_{jc} + M_{ij}\}. \quad (9.9)$$

A ring reducer passes summaries around a ring and computes

$$\bar{Y}_{ic} = \max_{r \leq R} Y_{ic}^{(r)}, \quad \bar{A}_{ic} = \bigcup_{r: Y_{ic}^{(r)} = \bar{Y}_{ic}} A_{ic}^{(r)}. \quad (9.10)$$

Theorem 9.5 (Exactness of tropical ring attention). *For any finite context length L_C , any partition $\mathcal{B}$ of $\{1, \dots, L_C\}$, any mask M , and any query/value channel (i, c) , the tropical ring summary satisfies*

$$\bar{Y}_{ic} = Y_{ic}, \quad \bar{A}_{ic} = A_{ic}. \quad (9.11)$$

The result is independent of the order in which blocks are reduced around the ring.

Proof. Because the blocks form a disjoint partition,

$$\max_{1 \leq j \leq L_C} z_j = \max_{r \leq R} \max_{j \in B_r} z_j \quad (9.12)$$

for $z_j = \langle Q_i, K_j \rangle + V_{jc} + M_{ij}$. The binary maximum is associative and commutative, so ring order cannot change the exact real-arithmetic result. A position j is a global maximizer if and only if it is a local maximizer in a block whose block value equals the global maximum. Hence the union rule gives exactly the global provenance set. $\square$

Corollary 9.6 (Streaming memory bound). *For m query rows, d -dimensional keys, c_o value channels, and block size B , exact tropical ring attention can be evaluated with working memory*

$$O(Bd + mc_o + mc_o R_{\text{live}}) \quad (9.13)$$

where R_{live} is the number of live block summaries retained for provenance. Without retaining full provenance, the working memory is $O(Bd + mc_o)$ plus the model state.

Proof. At any instant the evaluator needs the current block’s keys/values, the query table, and running maxima. Exact provenance requires storing either all tied maximizers or a bounded top- k summary. If only scores are needed, previous blocks can be discarded after their maxima are merged. $\square$

Definition 9.7 (Graph-sparse tropical ring attention). *Let $N_R(i) \subseteq \{1, \dots, L_C\}$ be the context positions within graph radius R of query position i after permission filtering. The sparse mask is*

$$M_{ij}^{\text{graph}} = \begin{cases} 0, & j \in N_R(i), \\ -\infty, & \text{otherwise.} \end{cases} \quad (9.14)$$

Sparse ring attention is exact attention over the graph-admissible neighborhood and has work proportional to $\sum_i |N_R(i)|$ rather than mL_C .

This distinction matters in a 10M-token setting. Full tropical scanning is useful for small m , global memory audits, or offline teacher passes. Online agents should usually retrieve a projected candidate set, expand it in the typed graph, and apply tropical ring attention to that induced subcontext plus a small set of global sentinel summaries.

9.4 Soft attention, tropical attention, and answer probability

Tropical ring attention is not a replacement for all soft attention. It is the exact primitive for active evidence and dynamic programming. Ordinary next-byte prediction still uses softmax or linear attention in the main model. The bridge is a side-channel: tropical summaries produce finite provenance codes and high-margin evidence features that condition ordinary logits.

Definition 9.8 (Tropical evidence side-channel). *Given ring summaries $\bar{Y}$ and provenance $\bar{A}$, define a compact side code*

$$\kappa_t = Q_{\text{code}}(\bar{Y}_t, \text{topk}(\bar{A}_t), \Delta_t, \text{types}(\bar{A}_t), \text{provenance}(\bar{A}_t)), \quad (9.15)$$

where Δ_t is the top-two tropical margin. The next-byte model is

$$q_\theta(b_t \mid b_{<t}, \mathfrak{M}_t) = \text{softmax}(\ell_\theta(b_{<t}) + R_\theta(\kappa_t, \mathfrak{M}_t)). \quad (9.16)$$

Proposition 9.9 (Margin stability of tropical side codes). *Assume all key, query, and value perturbations change every candidate score by at most ε . If the tropical top-two margin for (i, c) is $\Delta_{ic} > 2\varepsilon$, then the active provenance set for a unique maximizer is unchanged under the perturbation.*

Proof. Let j^* be the unique maximizer and j any competitor. Originally $z_{j^*} - z_j \geq \Delta_{ic}$. After perturbation, $\tilde{z}_{j^*} - \tilde{z}_j \geq \Delta_{ic} - 2\varepsilon > 0$. Hence j^* remains the unique maximizer. $\square$

High-margin side codes are exactly the kind of finite structure that can survive quantization. Low-margin decisions should be routed to the teacher, a verifier, or a larger context expansion because they are unstable under projection and quantization.

9.5 Learned projected retrieval for a 10M-token context

The vector database contains high-dimensional vectors $z_v \in \mathbb{R}^d$ and deployable projected vectors $u_v = P_\phi z_v \in \mathbb{R}^r$ with $r \ll d$. A retrieval head must learn P_ϕ so that positives remain separated from hard negatives after projection and quantization.

Definition 9.10 (Projected graph retrieval head). *For a query packet x with embedding z_x , projected query $u_x = P_\phi z_x$, and database record v , define the projected score*

$$S_\phi(x, v) = \langle P_\phi z_x, P_\phi z_v \rangle + \alpha \rho_\psi(x, v) + \beta g_\psi(x, v) - \gamma r_{\text{risk}}(x, v), \quad (9.17)$$

where ρ_ψ is a typed relation score, g_ψ is a graph-neighborhood score, and r_{risk} encodes permissions, stale provenance, unsafe tool surfaces, and behavior nonfaces. The ANN index stores $Q(P_\phi z_v)$, a quantized approximation of the projected vector.

Theorem 9.11 (Projection margin preservation). *Let p be a useful context record and n be a negative record for query x . Suppose the full score gap is*

$$\langle z_x, z_p \rangle - \langle z_x, z_n \rangle \geq \Gamma. \quad (9.18)$$

Assume the learned projection satisfies

$$|\langle P_\phi z_x, P_\phi z_v \rangle - \langle z_x, z_v \rangle| \leq \varepsilon_{\text{proj}} \quad (9.19)$$

for $v \in \{p, n\}$, and quantization adds at most ε_q score error per item. If

$$\Gamma > 2\varepsilon_{\text{proj}} + 2\varepsilon_q + \eta_{\text{graph}}, \quad (9.20)$$

where η_{graph} bounds graph/risk-score perturbation, then the projected quantized score still ranks p above n .

Proof. The projected unquantized gap is at least $\Gamma - 2\varepsilon_{\text{proj}}$. Quantization can reduce the positive score by ε_q and increase the negative score by ε_q , giving a gap at least $\Gamma - 2\varepsilon_{\text{proj}} - 2\varepsilon_q$. Graph and risk terms can reduce it by at most η_{graph} . The stated strict inequality leaves a positive gap. $\square$

The theorem gives a practical training rule: mine hard negatives whose full-score gap is small, then train the projection and quantizer to enlarge the deployed margin on exactly those pairs. Easy positives do not teach the retrieval head how to survive compression.

9.6 Quantized toric codes for context windows

ToricGT III uses quantization for retrieval vectors, chart codes, and side-channel summaries. The toric component is useful because it separates roles: a token block can be represented by a Cox degree, a one-dimensional-cone chart id, a semigroup code, a Stanley-Reisner compatibility bitmask, and a residual vector. These codes are small and compositional.

Definition 9.12 (Toric context code). *A toric context code for a record v is*

$$\chi(v) = (q_v, \deg_{\text{Cox}}(v), \sigma(v), H(v), \Delta(v), \pi(v), s(v)), \quad (9.21)$$

where q_v is a product-quantized retrieval code, $\deg_{\text{Cox}}(v)$ is a multidegree, $\sigma(v)$ is a cone or normal-fan cell id, $H(v)$ is a Hilbert-basis or semigroup-generator mask, $\Delta(v)$ is a filtered-complex signature, $\pi(v)$ is permission/provenance metadata, and $s(v)$ is a small sheaf-section summary.

Definition 9.13 (Toric code length). *The deployable code length of a context package S is*

$$\text{bits}(\chi(S)) = \sum_{v \in S} \text{bits}(\chi(v)) + \text{bits}(E_S) + \text{bits}(\text{codebooks}) + \text{bits}(\text{projection}). \quad (9.22)$$

The code is admissible only if this length is counted in the artifact or runtime side-information budget.

Theorem 9.14 (Side-information criterion for context utility). *Let Y be the future answer bytes, X the visible prefix/problem, and C a retrieved context package whose serialized side code has expected length $B(C)$ bits. If the model can use C optimally and the code is prefix-visible, the expected net bit improvement is*

$$I(Y; C \mid X) - B(C). \quad (9.23)$$

Normalized by answer byte length $|Y|_{\text{byte}}$, the retrieved context can improve bits-per-byte only if

$$\frac{I(Y; C \mid X) - B(C)}{|Y|_{\text{byte}}} > 0. \quad (9.24)$$

Proof. Without C , the optimal expected code length for Y given X is $H(Y \mid X)$. With C available and encoded at cost $B(C)$, it is $H(Y \mid X, C) + B(C)$. The difference is $H(Y \mid X) - H(Y \mid X, C) - B(C) = I(Y; C \mid X) - B(C)$. Dividing by answer bytes gives the bits-per-byte criterion. $\square$

This theorem is the long-context version of the ToricGT deployment rule. A million extra tokens are not automatically helpful. They must carry more conditional information about future bytes, reasoning correctness, or behavior control than their representation costs.

9.7 Filtered simplicial persistence of a growing context

At iteration t , the canvas induces a filtered complex $\mathcal{K}_t(\lambda)$. Vertices are context blocks, MCP records, markdown sections, KG entities, and generated reasoning steps. Edges encode adjacency, citation, schema compatibility, temporal succession, graph relation, or retrieval coactivation. Higher simplices encode mutually compatible sets of evidence, tools, claims, or behavior factors.

Definition 9.15 (Growing context filtration). *A growing context filtration is a bifiltration*

$$\mathcal{K}_{t,\lambda} \subseteq \mathcal{K}_{t',\lambda'} \quad \text{whenever} \quad t \leq t' \text{ and } \lambda \leq \lambda', \quad (9.25)$$

where t is reasoning time and λ is an evidence, confidence, distance, or relevance threshold. The associated persistence module is

$$M_i(t, \lambda) = H_i(\mathcal{K}_{t,\lambda}; \mathbb{F}). \quad (9.26)$$

Proposition 9.16 (Monotone fill gives functorial persistence). *If the fill trajectory is monotone and thresholds are nested, then $(t, \lambda) \mapsto H_i(\mathcal{K}_{t,\lambda}; \mathbb{F})$ is a functor from the poset $([T] \times \Lambda, \leq)$ to finite-dimensional vector spaces.*

Proof. A monotone fill gives inclusions of complexes $\mathcal{K}_{t,\lambda} \hookrightarrow \mathcal{K}_{t',\lambda'}$ whenever $(t, \lambda) \leq (t', \lambda')$. Homology is functorial with respect to inclusions, and composition of inclusions maps to composition of homology maps. Thus the assignment is a persistence module. $\square$

When memory edits delete or rewrite blocks, inclusions may fail. Then the correct object is a zigzag module. This is common for MCP agents: an unsafe retrieval may be removed after a permission check; a stale memory may be replaced; a tool result may be superseded. ToricGT III logs both the monotone teacher view and the deployment zigzag view.

Definition 9.17 (Context persistence drift). *Let $D_i(t)$ be the persistence diagram or a finite barcode summary of $H_i(\mathcal{K}_{t,\lambda})$ over λ . The long-context topology drift is*

$$\mathcal{L}_{\text{pers-drift}} = \sum_{t < \tau_C} d_B(D_i(t), D_i(t+1))^2 \quad (9.27)$$

with d_B the bottleneck distance, or a differentiable sliced-Wasserstein surrogate.

A low drift is not always desirable. Hard problems require phase changes. The useful target is *explained* drift: topology may change when new evidence, a tool result, or a proof branch enters, but not because of unstable retrieval noise or style drift.

9.8 Sheaves and global sections over a long window

A 10M-token canvas is too large to trust by surface recency alone. It needs local-to-global checks. The sheaf language is a compact way to express them. A local section may be the factual content of a resource block, a tool schema, a table row, a proof obligation, a behavior profile, a citation cluster, or a memory entry. Adjacent sections must agree on overlaps.

Definition 9.18 (Context sheaf over a reasoning canvas). *Let $\mathcal{K}_t$ be the context complex. A context sheaf $\mathcal{F}_t$ assigns a vector space, lattice, finite set, or structured record $\mathcal{F}_t(\sigma)$ to each simplex $\sigma \in \mathcal{K}_t$, together with restriction maps*

$$\rho_{\tau\sigma} : \mathcal{F}_t(\sigma) \rightarrow \mathcal{F}_t(\tau), \quad \tau \subseteq \sigma, \quad (9.28)$$

such that $\rho_{v\tau}\rho_{\tau\sigma} = \rho_{v\sigma}$ whenever $v \subseteq \tau \subseteq \sigma$.

Definition 9.19 (Long-context global section residual). *Given predicted local sections $s_\sigma \in \mathcal{F}_t(\sigma)$, the Cech gluing residual is*

$$\mathcal{L}_{\check{C},t} = \sum_{\tau \subseteq \sigma} \|\rho_{\tau\sigma} s_\sigma - s_\tau\|^2. \quad (9.29)$$

For a trajectory, use the length-normalized loss

$$\mathcal{L}_{\check{C}}^{\text{long}} = \frac{1}{\max(1, L_C)} \sum_{t < \tau_C} \sum_{\tau \subseteq \sigma \in \mathcal{K}_t} \|\rho_{\tau\sigma} s_\sigma^t - s_\tau^t\|^2. \quad (9.30)$$

Theorem 9.20 (Zero Cech residual gives compatible context). *If $\mathcal{L}_{\check{C},t} = 0$, then the local sections form a compatible family. If $\mathcal{F}_t$ is a sheaf on the finite complex topology, this family determines a unique global section over the union of the covered context cells whenever the cover is jointly epimorphic in the usual sheaf sense.*

Proof. The equality $\rho_{\tau\sigma} s_\sigma = s_\tau$ for every inclusion is precisely the compatibility condition for local sections. The sheaf axiom says that compatible local sections over a cover glue uniquely to a global section. On a finite complex, the statement reduces to equalizer conditions over intersections; zero residual enforces those equalities. $\square$

This is useful for accuracy and tone. Factual claims should glue across cited resources; tool descriptions should glue with actual tool results; a user’s preferred register should glue across markdown memory and current answer; uncertainty should restrict consistently from global answer to individual claims.

9.9 Derived updates for MCP and memory edits

The context database is dynamic. A tool result may refine a claim, a markdown memory may split, and a knowledge-graph entity may merge with another. Derived language gives a precise error signal for whether an update preserves the intended information.

Definition 9.21 (Context update complex). *Let $C_t^\bullet$ be a cochain complex built from the context sheaf, persistence module, or free resolution at time t . A learned update produces a chain map candidate*

$$f_t : C_t^\bullet \rightarrow C_{t+1}^\bullet. \quad (9.31)$$

The update-cone residual is the homology or rank profile of the mapping cone $\text{Cone}(f_t)$.

Theorem 9.22 (Mapping-cone criterion). *A chain map $f : C^\bullet \rightarrow D^\bullet$ is a quasi-isomorphism if and only if $\text{Cone}(f)$ is acyclic.*

Proof. The mapping cone fits into a distinguished triangle $C^\bullet \rightarrow D^\bullet \rightarrow \text{Cone}(f) \rightarrow C^\bullet[1]$. The associated long exact sequence in cohomology shows that $H^k(f) : H^k(C) \rightarrow H^k(D)$ is an isomorphism for all k exactly when $H^k(\text{Cone}(f)) = 0$ for all k . $\square$

In practice, exact acyclicity is teacher-side. The deployable signal is a small vector: ranks, Betti summaries, cone homology dimensions, and a confidence margin. A memory rewrite is accepted when it improves utility while keeping the cone residual below the task-specific threshold.

9.10 Ergodic theory of long-context fill dynamics

The long-context fill process is a controlled dynamical system. Even when the raw context space is huge, the quantized deployment state is finite: projected vector codes, chart ids, behavior alcoves, permission masks, and small certificate summaries all lie in finite codebooks.

Definition 9.23 (Quantized fill state). *A quantized fill state is*

$$\bar{S}_t = (\bar{c}_t, \bar{g}_t, \bar{k}_t, \bar{f}_t, \bar{b}_t, \bar{r}_t), \quad (9.32)$$

where $\bar{c}_t$ is a context-pack code, $\bar{g}_t$ is a graph-neighborhood code, $\bar{k}_t$ is a filtered-complex signature, $\bar{f}_t$ is a sheaf-gluing signature, $\bar{b}_t$ is a behavior-corridor cell, and $\bar{r}_t$ is a retrieval/provenance signature. The learned policy and environment induce a Markov kernel on the finite set of possible $\bar{S}_t$.

Theorem 9.24 (Finite long-context recurrence). *For fixed codebooks, bounded database capacity, bounded context length $C_{\max}$, and deterministic decoding temperature 0, every quantized fill trajectory is eventually periodic. For stochastic decoding, the process is a finite Markov chain and admits at least one stationary distribution.*

Proof. In the deterministic case, a map from a finite set to itself must revisit a state; after the first repeated state, the trajectory cycles. In the stochastic case, the transition kernel is a finite stochastic matrix, which has a stationary distribution by compactness of the probability simplex. $\square$

Definition 9.25 (Long-context ergodic observables). *Important observables include*

$$\varphi_{\text{bits-per-byte}}(\bar{S}_t) = \text{local held-out bits-per-byte}, \quad (9.33)$$

$$\varphi_{\text{ret}}(\bar{S}_t) = \text{retrieval precision or recall at budget}, \quad (9.34)$$

$$\varphi_{\text{tone}}(\bar{S}_t) = \text{tone/personality corridor distance}, \quad (9.35)$$

$$\varphi_{\text{acc}}(\bar{S}_t) = \text{verifier disagreement or hallucination score}, \quad (9.36)$$

$$\varphi_{\text{risk}}(\bar{S}_t) = \text{permission, privacy, or tool-safety violation score}, \quad (9.37)$$

$$\varphi_{\text{topo}}(\bar{S}_t) = \text{persistence, Cech, or mapping-cone residual}. \quad (9.38)$$

The empirical long-run profile

$$\hat{\Phi}_T = \frac{1}{T} \sum_{t=0}^{T-1} (\varphi_{\text{bits-per-byte}}, \varphi_{\text{ret}}, \varphi_{\text{tone}}, \varphi_{\text{acc}}, \varphi_{\text{risk}}, \varphi_{\text{topo}})(\bar{S}_t) \quad (9.39)$$

selects cache policy. States with high recurrence, high information gain, and low risk become persistent compressed memory. States with high recurrence but high verifier or safety loss become forbidden or require confirmation.

9.11 Stopping, sufficiency, and context length control

A long context agent should not always fill the maximum window. The correct stopping rule balances answer certainty, context cost, and expected information gain.

Definition 9.26 (Context sufficiency score). *Let $p_\theta(a \mid q, \mathfrak{W}_t)$ be the answer distribution, v_ψ a verifier score, r_ψ a retrieval uncertainty score, and $c_t = L(\mathfrak{W}_t)/C_{\max}$. Define*

$$\text{Suff}_t = \alpha v_\psi(q, a_t, \mathfrak{W}_t) - \beta H(p_\theta(\cdot \mid q, \mathfrak{W}_t)) - \gamma r_\psi(q, \mathfrak{W}_t) - \delta c_t. \quad (9.40)$$

The answer gate fires when $\text{Suff}_t \geq \kappa$ and all mandatory schema, provenance, permission, and behavior checks pass.

Proposition 9.27 (One-step stopping criterion). *Let G_t be the expected future gain in answer log score from one more context-fill step and let K_t be the expected token/runtime/side-code cost measured in bits. A myopic stop is optimal among stop/continue for one additional step when*

$$G_t \leq K_t. \quad (9.41)$$

Proof. Continuing changes expected net code length by $K_t - G_t$. If this quantity is nonnegative, one more step does not improve the one-step net objective. If it is negative, the expected net code length decreases and continuing is myopically preferable. $\square$

This criterion should be trained as a calibration target: many hard tasks need far less than 10M tokens, while some multi-file MCP tasks require a large context. The agent should learn problem-dependent L_C rather than defaulting to maximum context.

9.12 Implementation for a 1–10M-token MCP/memory/KG agent

A practical system separates raw storage, graph retrieval, long-context streaming, and exact audits.

Listing 1: Long-context ToricGT III retrieval and fill loop.

```
def solve_with_long_context(problem, db, C_max=10_000_000):
    W = init_canvas(problem, C_max=C_max)
    state = init_reasoning_state(problem)
    while W.filled_tokens < C_max:
        q = query_packet(problem, W, state)
        # 1. Fast projected vector search.
        u = P_phi @ query_encoder(q)
        seeds = ann_index.topk(quantize(u), k=K_seed)
        # 2. Typed graph expansion with permission and safety filters.
        subgraph = expand_graph(db.graph, seeds, radius=R,
                                permissions=q.permissions,
                                nonfaces=SR_behavior_rules)
        # 3. Build streamed blocks from MCP, markdown, KG, and generated steps.
        blocks = pack_blocks(subgraph, W.remaining_budget,
                              scores=projected_scores(u, subgraph))
        # 4. Exact max-plus provenance over streamed blocks.
        trop = tropical_ring_attention(q.hidden_queries, stream(blocks),
                                       masks=prefix_permission_graph_masks)
        # 5. Ordinary decoder uses compact side-channel and selected text.
        delta, state = model_fill_step(problem, W, state, blocks, trop.side_code)
        W.append(delta)
        # 6. Stop when answer sufficiency beats expected context gain.
        if answer_gate(problem, W, state):
            break
    return render_answer(W, state), audit_trace(W, state)
```

The index should store at least five arrays: projected quantized vectors for ANN; CSR adjacency by typed relation; block offsets into raw text or MCP payloads; compact toric context codes; and permission/provenance masks. Exact sheaf, resolution, and persistence objects can be computed in a teacher-side audit process and distilled into small summaries.

9.13 Long-context training data construction

Training examples should include both short and long contexts. A record has the form

$$(q, \mathbb{D}_0, \{\mathfrak{W}_t\}_{t \leq \tau_C}, a, \mathcal{A}), \quad (9.42)$$

where $\mathcal{A}$ is an audit package containing retrieval positives, negatives, tool permissions, schema labels, evidence paths, markdown gluing labels, KG proof paths, verifier judgments, and optional algebraic certificates.

Useful synthetic families include: multi-hop knowledge-graph proofs with distractor communities; MCP tool-use tasks with schema-compatible and schema-incompatible tools; markdown project-memory tasks requiring a style profile plus factual recall; long mathematical derivations with tableaux or root-system subproblems; codebase navigation tasks where repository files are graph nodes; and safety tasks in which relevant information is present but permission or policy nonfaces block direct use.

The data curriculum should scale L_C in phases:

$$2^{12} \rightarrow 2^{14} \rightarrow 2^{16} \rightarrow 2^{18} \rightarrow 2^{20} \rightarrow 10^6 \rightarrow 10^7. \quad (9.43)$$

At each phase, keep a fixed short-context slice. A long-context method that improves 10M-token tasks but damages short ordinary text is not ready for deployment without a gate.

10 Long-context objective catalogue

The previous objective catalogue remains valid for graph databases. This section adds objectives designed specifically for a 1–10M-token iterative context canvas. Each objective is audit-first; it becomes train-time only after exact finite checks agree with the differentiable surrogate, and it becomes deployable only after held-out bits-per-byte or predeclared reasoning/control metrics improve under artifact accounting.

10.1 LC1. Context Canvas Bits-per-Byte Utility

Definition. For a context package $C_t \subset \mathbb{D}_t$ and answer bytes Y , define

$$U_{\text{canvas}}(C_t) = \frac{1}{|Y|_{\text{byte}}} \left[\log_2 \frac{q_\theta(Y | X, C_t)}{q_\theta(Y | X)} - \text{bits}(\chi(C_t)) \right]. \quad (10.1)$$

Why it helps. This is the direct long-context utility. It rejects context that is semantically attractive but entropy-increasing after serialization. **Integration.** Use it as an offline label for context packers and as a reward for GFlowNet context selection. Train on matched examples with and without C_t to avoid confounding.

10.2 LC2. Tropical Ring Provenance Margin Loss

Definition. Let $\Delta_{ic} = z_{ic}^{(1)} - z_{ic}^{(2)}$ be the top-two tropical margin. Use

$$\mathcal{L}_{\text{ring-margin}} = \sum_{i,c} \max(0, \gamma - \Delta_{ic}) w_{ic}, \quad (10.2)$$

where w_{ic} is high only for channels whose provenance is verified useful. **Why it helps.** High tropical margins make provenance robust under projection, block streaming, and quantization. **Integration.** Attach to evidence maxima, tool-choice maxima, active KG paths, and proof obligations. Do not apply globally; low margins are valid when the task is genuinely ambiguous.

10.3 LC3. Projected Retrieval Gap Preservation

Definition. For positives p and hard negatives n , train

$$\mathcal{L}_{\text{proj-gap}} = \max(0, \gamma - S_\phi(x, p) + S_\phi(x, n)). \quad (10.3)$$

Why it helps. The full database cannot be searched exactly at every step. The projected head must preserve useful ordering under dimension reduction. **Integration.** Mine negatives from same markdown file, same tool family, same KG relation type, and same behavior alcove.

10.4 LC4. Quantized Score Robustness

Definition. If $\hat{z}_v$ reconstructs $P_\phi z_v$ from a codebook, penalize

$$\mathcal{L}_{\text{qscore}} = \sum_{(x,v)} (\langle P_\phi z_x, P_\phi z_v \rangle - \langle \hat{z}_x, \hat{z}_v \rangle)^2. \quad (10.4)$$

Why it helps. A retrieval head that works only before quantization is not a deployable retrieval head. **Integration.** Use fake quantization during training and exact codebook reconstruction during validation.

10.5 LC5. Graph Expansion Recall Objective

Definition. Let $E_R(C)$ be the radius- R expansion of ANN seeds. Train the seed retriever so that

$$\mathcal{L}_{\text{expand}} = -\log \Pr[P(x) \cap E_R(C_k(x)) \neq \emptyset]. \quad (10.5)$$

Why it helps. For very large contexts, the seed need not be the exact record if it lands near the useful subgraph. **Integration.** Label positive neighborhoods, not only positive chunks. This is essential for KG and codebase retrieval.

10.6 LC6. MCP Capability Nonface Barrier

Definition. If Δ_{MCP} is the allowed capability complex, penalize forbidden coactivation

$$\mathcal{L}_{\text{MCP-SR}} = \sum_{F \notin \Delta_{\text{MCP}}} \prod_{i \in F} p_i. \quad (10.6)$$

Why it helps. Tool descriptors, private resources, and code-execution paths must not be combined merely because they are relevant. **Integration.** Use MCP permissions and tool-safety labels to build nonfaces; evaluate on adversarial tool-description tasks.

10.7 LC7. Markdown Cech Memory Gluing

Definition. For markdown blocks U_i and overlaps $U_i \cap U_j$, use

$$\mathcal{L}_{\text{md-}\check{C}} = \sum_{i < j} \|s_i|_{U_{ij}} - s_j|_{U_{ij}}\|^2. \quad (10.7)$$

Why it helps. Long-context agents often retrieve inconsistent versions of project memory. Cech gluing detects contradiction before generation. **Integration.** Build overlaps from headings, anchors, backlinks, timestamps, and citations.

10.8 LC8. KG Path Sheaf Consistency

Definition. For two KG paths p_1, p_2 supporting the same claim, compare transported sections:

$$\mathcal{L}_{\text{KG-path}} = \|T_{p_1} s_{\text{source}} - T_{p_2} s_{\text{source}}\|^2. \quad (10.8)$$

Why it helps. Multi-hop reasoning should not depend on arbitrary path choice unless the paths encode genuinely different evidence. **Integration.** Use relation-specific transports and train on symmetric, inverse, transitive, and typed relation families.

10.9 LC9. Long-Context Occupation Entropy Regularizer

Definition. Let $\hat{\mu}_T(c)$ be occupation of quantized context cells. Penalize collapse or chaos by

$$\mathcal{L}_{\text{occ}} = (H(\hat{\mu}_T) - H_{\text{target}})^2. \quad (10.9)$$

Why it helps. A long context agent should reuse stable states without collapsing to the same retrieval pattern for every task. **Integration.** Estimate on task slices; set H_{target} differently for deterministic algebra tasks and exploratory research tasks.

10.10 LC10. Recurrence Utility Cache Loss

Definition. For a context cell c , define cache utility

$$U_{\text{cache}}(c) = \hat{\mu}(c) \Delta_{\text{bits-per-byte}}(c) - \lambda_{\text{risk}} R(c) - \lambda_{\text{bytes}} \text{bits}(c). \quad (10.10)$$

Why it helps. Frequently reused helpful states deserve compact persistent codes; frequently reused harmful states should be gated. **Integration.** Use this to promote records from raw memory to quantized fast memory.

10.11 LC11. Context Packing Submodularity Loss

Definition. For learned utility $U(S)$, penalize violations of diminishing returns:

$$\mathcal{L}_{\text{submod}} = \sum_{A \subseteq B, v \notin B} \max(0, \Delta_v(B) - \Delta_v(A))^2. \quad (10.11)$$

Why it helps. Approximate submodularity makes greedy context packing reliable under token budgets. **Integration.** Sample small sets A, B from candidate packs; do not require exact submodularity for all context.

10.12 LC12. Reasoning Fill Monotonicity and Zigzag Alarm

Definition. For monotone phases, penalize deletion of required support:

$$\mathcal{L}_{\text{fill}} = \sum_t \|\mathbf{1}_{I_t} \odot (1 - \mathbf{1}_{I_{t+1}})\|_1. \quad (10.12)$$

For edit phases, log a zigzag alarm when deletion changes persistence beyond threshold. **Why it helps.** The model should not silently forget evidence already used in a proof. **Integration.** Allow explicit memory rewrite actions, but require mapping-cone and verifier audits.

10.13 LC13. Derived Update Cone Loss

Definition. For update chain map $f_t : C_t^\bullet \rightarrow C_{t+1}^\bullet$, use

$$\mathcal{L}_{\text{cone}} = \sum_k \dim H^k(\text{Cone}(f_t)) \quad (10.13)$$

when exact, or a differentiable rank surrogate. **Why it helps.** It distinguishes information-preserving summarization from destructive memory rewrite. **Integration.** Use exact finite-field audits offline and low-rank surrogates online.

10.14 LC14. Persistence Event Explanation Loss

Definition. Let e_t be an event label: new evidence, tool result, contradiction, permission removal, or style memory. Train

$$\mathcal{L}_{\text{event}} = \text{CE}(e_t, \hat{e}_t) + \lambda \|\Delta D_t - R_{e_t}\|^2, \quad (10.14)$$

where ΔD_t is barcode change and R_{e_t} is the expected pattern. **Why it helps.** Topological drift is useful only when explained by an actual reasoning event. **Integration.** Label synthetic long-context tasks with known evidence arrivals.

10.15 LC15. Tone/Personality Global Section Loss

Definition. Let b_σ be behavior sections over context cells. Use

$$\mathcal{L}_{\text{tone-global}} = \sum_{\tau \subseteq \sigma} \|\rho_{\tau\sigma} b_\sigma - b_\tau\|^2 + \lambda \sum_t \text{dist}(b_t, K_{\text{task}})^2. \quad (10.15)$$

Why it helps. Tone and personality drift across a 10M-token trajectory can degrade user trust and accuracy. **Integration.** Apply to style memories, current user instructions, safety state, and answer sections.

10.16 LC16. Evidence Boundary Accuracy Loss

Definition. For each answer claim a_j , let ∂a_j be its required evidence boundary. Penalize missing support:

$$\mathcal{L}_{\text{evidence}} = \sum_j \max(0, \gamma - \text{support}(\partial a_j, \mathfrak{W}_{\tau_C})). \quad (10.16)$$

Why it helps. Long context increases hallucination risk unless claims are tied to retrieved evidence. **Integration.** Use verifier-generated claim/evidence pairs and KG path labels.

10.17 LC17. Tool-Call Schema Degree Loss

Definition. Assign each tool-call argument a Cox-like degree. Penalize degree mismatch:

$$\mathcal{L}_{\text{schema-deg}} = \sum_a \|\deg(a) - \deg_{\text{schema}}(a)\|_1 + \lambda \mathbf{1}\{\text{invalid JSON/schema}\}. \quad (10.17)$$

Why it helps. It makes tool validity a typed algebraic constraint rather than a surface format afterthought. **Integration.** Train on MCP tool schemas with adversarial near-valid calls.

10.18 LC18. Privacy and Permission Ideal Loss

Definition. Let I_{priv} be an ideal or monomial nonface set generated by forbidden combinations of private resources, tools, and outputs. Use

$$\mathcal{L}_{\text{priv}} = \sum_{x^F \in I_{\text{priv}}} \prod_{i \in F} p_i. \quad (10.18)$$

Why it helps. Relevance does not imply permission. This loss blocks unsafe context joins. **Integration.** Use host/client/server permission boundaries and consent labels.

10.19 LC19. Tropical Segment-Tree Distillation

Definition. Let T_{full} be full tropical ring output and T_{seg} be a compressed segment tree of block summaries. Train

$$\mathcal{L}_{\text{seg}} = \|T_{\text{full}} - T_{\text{seg}}\|^2 + \lambda d_{\text{prov}}(A_{\text{full}}, A_{\text{seg}}). \quad (10.19)$$

Why it helps. Segment summaries let a 10M-token context expose global evidence without rescanning every token. **Integration.** Use teacher scans to train persistent block summaries.

10.20 LC20. Answer Stopping-Time Efficiency

Definition. With terminal time τ_C , answer quality Q , and context cost L_C , train

$$\mathcal{L}_{\text{stop}} = -Q(a, \mathfrak{M}_{\tau_C}) + \lambda \frac{L_C}{C_{\text{max}}} + \eta \max(0, \kappa - \text{Suff}_{\tau_C}). \quad (10.20)$$

Why it helps. The agent should use 10M tokens only when the problem requires them. **Integration.** Train on tasks with known minimal evidence sets and evaluate quality as a function of allowed context length.

11 Training objectives and losses

11.1 Base objective

The base objective is always the prequential byte loss, written explicitly as bits-per-byte:

$$\mathcal{L}_{\text{bits-per-byte}}(\theta) = -\frac{1}{T} \sum_{t=1}^T \log_2 q\theta(b_t \mid b_{<t}, \text{retrieve}_{<t}). \quad (11.1)$$

Retrieval must be prefix-safe. Auxiliary losses are staged:

$$\mathcal{L} = \mathcal{L}_{\text{bits-per-byte}} + \lambda_r \mathcal{L}_{\text{retr}} + \lambda_g \mathcal{L}_{\text{graph}} + \lambda_s \mathcal{L}_{\text{sheaf}} + \lambda_q \mathcal{L}_{\text{quant}} + \sum_j \lambda_j \mathcal{L}_j. \quad (11.2)$$

The deployment gate is

$$\lambda_j > 0 \Rightarrow \Delta \text{bits-per-byte}_j > -\delta, \quad \text{audit}_j = 1, \quad \text{artifact}_j \leq A_{\text{max}}, \quad \text{security}_j = \text{pass}. \quad (11.3)$$

11.2 Core retrieval objective

For a query x , positives $P(x)$, hard negatives $N(x)$, and a candidate set $C = P(x) \cup N(x)$, the retrieval objective is

$$\mathcal{L}_{\text{retr}}(x) = -\log \frac{\sum_{v \in P(x)} \exp S(x, v)/\tau}{\sum_{u \in C} \exp S(x, u)/\tau}. \quad (11.4)$$

Graph smoothness and typed relation calibration are

$$\mathcal{L}_{\text{lap}} = \sum_{(u,v) \in E} w_{uv} \|z_u - R_{\tau(u,v)} z_v\|^2, \quad (11.5)$$

$$\mathcal{L}_{\text{rel}} = \sum_{(u,\tau,v)} \max(0, \gamma + \|z_u + R_{\tau} - z_v\|^2 - \|z_u + R_{\tau'} - z_{v'}\|^2). \quad (11.6)$$

The graph-structured objective is

$$\mathcal{L}_{\text{gdb}} = \mathcal{L}_{\text{retr}} + \lambda_{\text{lap}} \mathcal{L}_{\text{lap}} + \lambda_{\text{rel}} \mathcal{L}_{\text{rel}} + \lambda_{\check{C}} \mathcal{L}_{\check{C}} + \lambda_{\text{form}} \mathcal{L}_{\text{form}} + \lambda_{\text{tq}} \mathcal{L}_{\text{tq}}. \quad (11.7)$$

11.3 Forty objectives for ToricGT III

The catalogue has two parts. Part I adapts the best ToricGT II objectives to MCP/memory/KG retrieval. Part II introduces objectives native to graph-structured vector databases and protocol-safe context systems. Each item includes a mathematical object, loss or metric, reason it can improve bits-per-byte or reasoning, and integration notes.

ID	Objective	Primary role in ToricGT III
A1	Context side-information bits-per-byte gate	Promote MCP/memory/KG certificates only when prefix-visible retrieval improves held-out bits-per-byte or verified reasoning.
A2	MCP certificate conditional information	Estimate $I(B_t; Z_t \mid \mathcal{F}_t)$ for tool/resource/memory codes; suppress decorative retrieval metadata.
A3	Protocol active-cell entropy margin	Stabilize tool/resource/prompt active regions in projected retrieval space.
A4	Saturated semigroup memory code	Remove missing-code holes in quantized metadata and context-role algebra.
A5	Hilbert-basis context primitive sparsity	Store reusable minimal context primitives rather than redundant long traces.
A6	Toric-ideal KG path consistency	Enforce equivalent relation paths and schema compositions to score similarly.
A7	Cox-degree role conservation	Preserve evidence, verifier, style, memory, tool, and answer-commitment degrees through retrieval.
A8	Cartier support-function retrieval slope	Share local ranking slopes across equivalent context charts.
A9	Moment-map context summary quantizer	Convert active retrieval geometry into a compact moment vector.
A10	Stanley-Reisner forbidden context nonfaces	Penalize incompatible coactivations such as low evidence plus high confidence or unsafe tool plus no consent.
A11	Cech markdown-memory gluing	Detect and penalize incompatible local memory sections.
A12	Vector-bundle behavior transport	Maintain coherent tone, persona, rigor, and refusal axes across retrieved contexts.
A13	Filtration-bundle expert retrieval	Route query types to specialized retrieval subbundles while sharing a compact base representation.
A14	Closed differential-form evidence flow	Enforce path-independent evidence accumulation in retrieved support graphs.
A15	Secondary-fan schema migration	Control schema wall crossings when MCP servers or memory formats change.
A16	Gale-dual tool/resource pairing	Pair tools with dual resources or diagnostics to improve transfer and audit.
A17	Young-tableau ontology compression	Canonicalize hierarchical entity schemas, task decompositions, and output templates.
A18	Littlewood-Richardson context composition	Supervise branch-merge retrieval plans by associative combinatorial composition.
A19	Derived memory-update chain map	Measure database updates by mapping-cone residuals.
A20	Ergodic context Pareto controller	Gate long-run tradeoffs among bits-per-byte, retrieval, verifier, latency, safety, and artifact bytes.
N1	MCP trace graph reconstruction	Train embeddings to reconstruct event types, call-return edges, schemas, and permissions.
N2	Markdown heading sheaf exactness	Make local summaries and retrieved section data glue across heading overlaps.
N3	Tool/resource schema preservation	Ensure projected vectors preserve JSON-schema and output-schema compatibility.
N4	Graph-structured ANN recall	Train projection and quantization for high recall after graph expansion.

ID	Objective	Primary role in ToricGT III
N5	Projection distortion margin loss	Penalize hard-pair inner-product distortion near retrieval decision boundaries.
N6	Quantized adjacency-aware retrieval	Train vector codes jointly with typed adjacency and path features.
N7	Context-packing submodular utility	Select compact context packages maximizing predicted utility under token and artifact budgets.
N8	Provenance residue safety loss	Treat unresolved provenance or consent obligations as logarithmic boundary residues.
N9	Tool-call return cycle consistency	Check that tool outputs answer the schema and semantic intent of the call.
N10	Prompt-template section distillation	Distill reusable prompt templates as sections over task charts.
N11	KG path-algebra normal form	Canonicalize relation paths and suppress spurious equivalent paths.
N12	Simplicial coactivation memory nonfaces	Use a Stanley-Reisner ideal to reject incoherent retrieved memory packages.
N13	Local-cohomology conflict alarm	Use local obstruction classes to flag contradictions and missing context.
N14	MCP cache spectral-gap controller	Tune cache refresh and eviction using mixing and recurrence diagnostics.
N15	Retrieval trace entropy-rate loss	Reduce irrelevant retrieval oscillation without collapsing useful exploration.
N16	Memory update mapping-cone residual	Quantify whether edits preserve or intentionally change derived context structure.
N17	Derived index compaction	Compress Betti/resolution signatures for fast retrieval-side comparison.
N18	Toroidal periodic context cache	Use phase codes for recurring tasks, cron-like tools, and periodic memories.
N19	Behavior-corridor memory sheaf	Store tone/personality and accuracy controls as sheaf sections over memories.
N20	Artifact-budget context controller	Decide what retrieval heads, codes, and certificates enter the deployable artifact.

11.4 Part I: adapted ToricGT objectives

A1. Context side-information bits-per-byte gate. Object. Prefix-safe MCP or memory certificate Z_t , such as a tool schema code, sheaf-gluing flag, graph path signature, or quantized toric chart code. **Loss/metric.**

$$\Delta\text{bits-per-byte}(Z) = \frac{1}{T} \sum_t \log_2 \frac{q_\theta(b_t | \mathcal{F}_t, Z_t)}{q_0(b_t | \mathcal{F}_t)} - \lambda_A \text{bytes}(Z) - \lambda_L \text{latency}(Z). \quad (11.8)$$

Why it helps. This is the promotion controller. It prevents MCP machinery from becoming a decorative retrieval stack. A certificate is useful only if it lowers conditional byte entropy or a predeclared verified-reasoning slice without exceeding artifact and latency budgets. **Integration.** Run base, shuffled-certificate, teacher-certificate, and quantized-student ablations. Promote only if exact audits pass and held-out bits-per-byte is nondegrading.

A2. MCP certificate conditional information. Object. A finite code Z for retrieved context: source, schema, tool, heading, KG motif, persistence barcode, or Cech status. **Metric.** Estimate conditional information with a held-out classifier or likelihood ratio:

$$\hat{I}(B; Z | C) = \frac{1}{n} \sum_i \log_2 \frac{\hat{q}(b_i | c_i, z_i)}{\hat{q}(b_i | c_i)}. \quad (11.9)$$

Why it helps. It tells whether the retrieval structure predicts the next bytes, not whether it looks coherent in isolation. **Integration.** Use it for probe selection and logging. Do not backpropagate aggressively until matched-seed bits-per-byte lift is observed.

A3. Protocol active-cell entropy margin. Object. Normal-fan cells in projected query space, induced by affine retrieval candidates. **Loss.**

$$\mathcal{L}_{\text{cell}} = H(C_t | \text{task}) + \lambda \mathbb{E} \max(0, \gamma - m_t), \quad (11.10)$$

where C_t is active cell id and m_t is top-two cell margin. **Why it helps.** Stable cells make tool/resource selection predictable and reduce context jitter. **Integration.** Audit against exact cell ids after rationalizing candidate slopes; reject if entropy drops by collapse rather than better retrieval.

A4. Saturated semigroup memory code. Object. Metadata codes in an affine semigroup $S_A = \mathbb{N}A$. **Loss.**

$$\mathcal{L}_{\text{sat}} = \text{dist}(\hat{c}, S_A^{\text{sat}})^2 + \lambda \mathbf{1}\{\hat{c} \in S_A^{\text{sat}} \setminus S_A\}. \quad (11.11)$$

Why it helps. Missing quantized combinations are memory holes: the model can represent pieces but not their valid combination. Saturation normalizes the codebook. **Integration.** Compute holes offline by finite Hilbert-basis or bounded search; train a small code head.

A5. Hilbert-basis context primitive sparsity. Object. Minimal semigroup generators of valid context codes. **Loss.**

$$\mathcal{L}_{\text{Hilb}} = \|a\|_0^{\text{soft}} + \lambda \|c - \sum_i a_i h_i\|^2. \quad (11.12)$$

Why it helps. Hilbert-basis primitives are reusable. They reduce redundant memory phrases and shorten retrieved context. **Integration.** Learn sparse codes over primitives for resource/tool/mark-down/KG motifs; promote only if context-packing utility improves.

A6. Toric-ideal KG path consistency. **Object.** Equivalent typed paths in a knowledge graph, $p \sim p'$ when their relation composition has the same normal form. **Loss.**

$$\mathcal{L}_{\text{path}} = \sum_{p \sim p'} \|R_p z_u - R_{p'} z_u\|^2 + \lambda \sum_{x^u - x^v \in I_A} \|u \cdot \ell - v \cdot \ell\|^2. \quad (11.13)$$

Why it helps. Equivalent evidence routes should not produce incompatible retrieval rankings. **Integration.** Generate low-degree path relations from schema rules and graph mining; verify on held-out KG queries.

A7. Cox-degree role conservation. **Object.** Multidegrees for context roles: evidence, verifier, memory, safety, style, tool, resource, prompt. **Loss.**

$$\mathcal{L}_{\text{Cox}} = \sum_{(u,v) \in E} \|\deg v - \deg u - \deg(e_{uv})\|_1. \quad (11.14)$$

Why it helps. Role conservation prevents evidence vectors from being silently used as style vectors or tool permission vectors. **Integration.** Store role degrees as tiny integers; use them in retrieval reranking and invalid-package rejection.

A8. Cartier support-function retrieval slope. **Object.** Piecewise-linear ranking functions on retrieval cells. **Loss.**

$$\mathcal{L}_{\text{Cart}} = \sum_{\sigma} \sum_{v \in \sigma} (S(q, v) - \langle m_{\sigma}, \phi(q, v) \rangle - c_{\sigma})^2 + \lambda \sum_{\tau} \|m_{\sigma} - m_{\sigma'} - b_{\tau}\|^2. \quad (11.15)$$

Why it helps. It makes retrieval rules share slopes across equivalent charts, improving compression and stability. **Integration.** Fit rational slopes on sampled cells; treat wall bends as explanations of tool or memory switches.

A9. Moment-map context summary quantizer. **Object.** Softmax-weighted convex combination of active retrieval candidate exponents. **Loss.**

$$\mu_{\beta}(q) = \frac{\sum_i e^{\beta s_i(q)} a_i}{\sum_i e^{\beta s_i(q)}}, \quad \mathcal{L}_{\text{mom}} = \|\mu_{\beta}(q) - \text{deq}(Q(\mu_{\beta}(q)))\|^2. \quad (11.16)$$

Why it helps. A small moment vector summarizes a large active context neighborhood. **Integration.** Serialize only the quantized moment when it improves reranking or answer likelihood.

A10. Stanley-Reisner forbidden context nonfaces. **Object.** A simplicial complex of allowed context coactivations. **Loss.**

$$\mathcal{L}_{\text{SR}} = \sum_{F \notin \Delta} \prod_{v \in F} p_v. \quad (11.17)$$

Why it helps. It cheaply forbids dangerous or incoherent packages: stale memory with high confidence, unsafe tool with no consent, answer commitment without verifier. **Integration.** Maintain nonface lists as policy and data-quality rules; use exact audit on sampled retrieval packages.

A11. Cech markdown-memory gluing. **Object.** Memory sheaf over heading and link covers. **Loss.** Use $\mathcal{L}_{\tilde{C}}$ from Section 3. **Why it helps.** A model that glues local memory sections correctly emits fewer contradictions and fewer overconfident stale statements. **Integration.** Build heading covers offline; update sections after memory edits; expose obstruction summaries to the retrieval head.

A12. Vector-bundle behavior transport. **Object.** Local frames for behavior axes over contexts. **Loss.** Use $\mathcal{L}_{\text{bundle}}$. **Why it helps.** It controls tone, persona, rigor, and refusal policy across retrieved memories without using one global style vector. **Integration.** Train on contrastive behavior data and verifier-calibrated answer traces; demote if ordinary prose bits-per-byte regresses.

A13. Filtration-bundle expert retrieval. **Object.** A bundle whose fibers are expert subspaces indexed by task/filter coordinates. **Loss.**

$$\mathcal{L}_{\text{exp}} = \sum_i \|E_i(q) - g_{ij}E_j(q)\|^2 + \lambda H(\pi_{\text{expert}} \mid q). \quad (11.18)$$

Why it helps. It specializes retrieval for code, math, literature, tools, and memory without storing fully separate indexes. **Integration.** Share base projection; use low-rank adapters per fiber.

A14. Closed differential-form evidence flow. **Object.** Evidence 1-form on retrieved graph. **Loss.** Use $\mathcal{L}_{\text{form}}$. **Why it helps.** It detects path-dependent contradictions and encourages stable evidence accumulation. **Integration.** Apply to triangles and short cycles in retrieved subgraphs; expose nonzero circulation as a caution signal.

A15. Secondary-fan schema migration. **Object.** Chambers of schema versions, server capabilities, or database layouts. **Loss.**

$$\mathcal{L}_{\text{schema}} = \sum_v \text{CE}(c_v, \hat{c}_v) + \lambda \sum_{\text{walls}} \text{drift}(\text{normalform}_{\text{old}}, \text{normalform}_{\text{new}}). \quad (11.19)$$

Why it helps. MCP servers evolve. A secondary-fan-style chamber code lets the retriever cross schema walls intentionally. **Integration.** Use migration tests as synthetic data.

A16. Gale-dual tool/resource pairing. **Object.** Dual relation between tool actions and resource evidence, or queries and diagnostics. **Loss.**

$$\mathcal{L}_{\text{Gale}} = \|G_{\theta}s_{\text{tool}} - s_{\text{resource}}\|^2 + \|G_{\theta}^{-1}s_{\text{resource}} - s_{\text{tool}}\|^2. \quad (11.20)$$

Why it helps. It improves transfer: a tool call and the resource that verifies it should have compatible signatures. **Integration.** Build pairs from logs where a tool result is later cited by a correct answer.

A17. Young-tableau ontology compression. **Object.** Semistandard tableaux encoding hierarchical schemas, entity types, or decomposition plans. **Loss.**

$$\mathcal{L}_{\text{tab}} = \text{CE}(T, \hat{T}) + \lambda \text{Inv}(\hat{T}) + \lambda_w \|\text{wt}(\hat{T}) - \text{wt}(T)\|_1. \quad (11.21)$$

Why it helps. Tableau normal forms canonicalize repeated ontology patterns and reduce prompt/schema verbosity. **Integration.** Use for enterprise schemas, biomedical ontologies, theorem-plan templates, and task decomposition.

A18. Littlewood-Richardson context composition. **Object.** Branch-merge composition of retrieved contexts. **Loss.**

$$\mathcal{L}_{\text{LR}} = -\log p_{\theta}(\lambda \mid \mu, \nu) + \lambda_{\text{assoc}} \|C_{(a \otimes b) \otimes c} - C_{a \otimes (b \otimes c)}\|^2. \quad (11.22)$$

Why it helps. It makes retrieval-plan composition associative and compact. **Integration.** Apply to multi-hop retrieval where two evidence branches must merge before answer commitment.

A19. Derived memory-update chain map. **Object.** Chain maps between derived context objects before and after update. **Loss.**

$$\mathcal{L}_{\text{cone}} = \|\partial_{t+1}F - F\partial_t\|_F^2 + \lambda \dim H(\text{Cone}(F))_{\text{unexpected}}. \quad (11.23)$$

Why it helps. It separates valid memory refinements from destructive edits. **Integration.** Use on memory updates, resource refreshes, schema migrations, and KG imports.

A20. Ergodic context Pareto controller. **Object.** Long-run averages of bits-per-byte, retrieval quality, verifier score, latency, safety, and artifact bytes. **Controller.**

$$\lambda_j(t+1) = \Pi_{[0, \lambda_{\max}]} \left(\lambda_j(t) + \eta(\bar{g}_j(t) - g_j^*) \right). \quad (11.24)$$

Why it helps. It prevents local retrieval wins from damaging long-run behavior. **Integration.** Estimate on rolling validation streams and require slice-level nondegradation.

11.5 Part II: new MCP/memory/KG-native objectives

N1. MCP trace graph reconstruction. Object. Protocol graph G_{mcp} . **Loss.**

$$\mathcal{L}_{\text{mcp-rec}} = \text{CE}(\tau_V, \hat{\tau}_V) + \text{CE}(\tau_E, \hat{\tau}_E) + \sum_{(u,v)} \text{BCE}(A_{uv}, \hat{A}_{uv}) + \lambda \mathcal{L}_{\text{schema}}. \quad (11.25)$$

Why it helps. A retrieval vector that cannot reconstruct call-return, resource, schema, and permission edges is unsafe and low value. **Integration.** Train from MCP logs with negative edges sampled by time and type.

N2. Markdown heading sheaf exactness. Object. Heading cover of markdown memory. **Loss.**

$$\mathcal{L}_{\text{md}} = \sum_{i < j} \|s_i|_{ij} - s_j|_{ij}\|^2 + \lambda \sum_i \text{CE}(\text{heading}_i, \hat{h}_i). \quad (11.26)$$

Why it helps. It makes long-term memory retrieval local-to-global rather than string-matching. **Integration.** Apply to every saved memory file and project notes; cache local summaries by heading hash.

N3. Tool/resource schema preservation. Object. JSON-schema and output-schema labels for tools and resources. **Loss.**

$$\mathcal{L}_{\text{schema}} = \text{CE}(\text{valid}, \hat{v}) + \sum_f \text{CE}(\text{type}_f, \widehat{\text{type}}_f) + \lambda \text{BCE}(\text{required}_f, \hat{r}_f). \quad (11.27)$$

Why it helps. It reduces invalid tool calls and improves self-correction after tool errors. **Integration.** Use tool definitions, structured outputs, and validation errors as supervised data.

N4. Graph-structured ANN recall. Object. Candidate generation after projected ANN and graph expansion. **Metric.**

$$\text{GraphRecall}@k = \Pr[P(x) \cap \overline{C}_k(x) \neq \emptyset]. \quad (11.28)$$

Why it helps. Fast ANN may miss the exact node, but graph expansion can recover useful neighbors. **Integration.** Optimize projection for recall after expansion, not raw vector recall only.

N5. Projection distortion margin loss. Object. Learned projection P_ϕ . **Loss.**

$$\mathcal{L}_{\text{margin}} = \sum_{(q, v^+, v^-)} \max(0, \gamma - [\tilde{s}(q, v^+) - \tilde{s}(q, v^-)]) + \lambda \mathcal{L}_{\text{dist}}. \quad (11.29)$$

Why it helps. It preserves retrieval order in low dimension. **Integration.** Mine hard triples from exact high-dimensional reranker logs.

N6. Quantized adjacency-aware retrieval. **Object.** Quantized vectors and typed adjacency. **Loss.**

$$\mathcal{L}_{\text{qadj}} = \mathcal{L}_{\text{retr}}(Q(Pz)) + \lambda \sum_{(u,v) \in E} \|\text{deq}(q_u) - R_{uv} \text{deq}(q_v)\|^2. \quad (11.30)$$

Why it helps. It aligns compression codes with graph structure, improving small-index retrieval. **Integration.** Train with fake quantization; audit exact quantized export.

N7. Context-packing submodular utility. **Object.** Set function $U(S)$ for a context package. **Loss.**

$$\mathcal{L}_{\text{pack}} = -\mathbb{E}_{S \sim \pi_\theta} [U(S) - \lambda_{\text{tok}}|S| - \lambda_{\text{risk}}R(S)]. \quad (11.31)$$

Why it helps. It trains the model to include context that changes answers, not merely context that is similar. **Integration.** Use differentiable greedy or GFlowNet-style set sampling with verifier reward.

N8. Provenance residue safety loss. **Object.** Boundary divisors for missing consent, stale source, unverifiable result, or unsafe tool. **Loss.**

$$\mathcal{L}_{\text{prov}} = \sum_{\rho \in \Sigma(1)} \alpha_\rho |\text{Res}_{D_\rho}(\omega_{\text{answer}}) - r_\rho^*|^2. \quad (11.32)$$

Why it helps. Boundary residues become explicit caution or refusal signals. **Integration.** Attach residues to retrieved package summaries and answer-style controller.

N9. Tool-call return cycle consistency. **Object.** Cycle query $\rightarrow$ tool call $\rightarrow$ result $\rightarrow$ answer. **Loss.**

$$\mathcal{L}_{\text{toolcycle}} = \|\psi_{\text{intent}}(q) - \psi_{\text{result}}(r)\|^2 + \lambda \text{CE}(\text{answers_schema}, \hat{y}). \quad (11.33)$$

Why it helps. It prevents using tool outputs that do not answer the call. **Integration.** Sample tool traces with known success and failure results.

N10. Prompt-template section distillation. **Object.** Prompt templates as local sections over task charts. **Loss.**

$$\mathcal{L}_{\text{prompt}} = \sum_i \|s_i - \hat{s}_i\|^2 + \lambda \text{CE}(\text{template_id}, \widehat{\text{template}}). \quad (11.34)$$

Why it helps. It makes reusable prompts compact and task-local. **Integration.** Store prompt templates as MCP prompt resources with section metadata.

N11. KG path-algebra normal form. **Object.** Path algebra quotient by schema relations. **Loss.**

$$\mathcal{L}_{\text{nf}} = \text{CE}(\text{NF}(p), \widehat{\text{NF}}(p)) + \lambda \|R_p - R_{\text{NF}(p)}\|^2. \quad (11.35)$$

Why it helps. It canonicalizes multi-hop evidence and reduces spurious path explosion. **Integration.** Compute normal forms for ontology rules and database relations.

N12. Simplicial coactivation memory nonfaces. **Object.** Forbidden memory packages. **Loss.** Same as $\mathcal{L}_{\text{SR}}$, specialized to memory. **Why it helps.** It prevents retrieving mutually contradictory or policy-incompatible memories. **Integration.** Build nonfaces from contradiction detectors, user consent, and verifier failures.

N13. Local-cohomology conflict alarm. **Object.** Local obstructions supported near problematic context subsets. **Metric.**

$$\text{Conflict}(S) = \dim H_S^1(\mathcal{F}) + \lambda \dim H_S^0(\mathcal{F}/\mathcal{F}_{\text{trusted}}). \quad (11.36)$$

Why it helps. It localizes contradictions instead of globally distrusting the whole memory. **Integration.** Run exact sidecar audits on sampled retrieval packages.

N14. MCP cache spectral-gap controller. **Object.** Markov transition matrix among quantized context cells. **Metric.**

$$\text{gap} = 1 - |\lambda_2(P)|. \quad (11.37)$$

Why it helps. A tiny gap suggests sticky stale context; an excessive gap may indicate unstable retrieval. **Integration.** Use gap estimates to tune cache refresh, TTL, and eviction.

N15. Retrieval trace entropy-rate loss. **Object.** Symbolic sequence of retrieved cells. **Loss.**

$$\mathcal{L}_{\text{hrate}} = \left(\hat{h}_m - h_m^* \right)^2 + \lambda \text{Var}(\hat{h}_m). \quad (11.38)$$

Why it helps. It reduces context oscillation while preserving adaptive diversity. **Integration.** Apply to long conversations and repeated tasks.

N16. Memory update mapping-cone residual. **Object.** Derived context objects before and after edit. **Loss.** Same as $\mathcal{L}_{\text{cone}}$, with labels for intended edit type. **Why it helps.** It distinguishes harmless refactors from semantic memory changes. **Integration.** Use when the agent writes or summarizes memory.

N17. Derived index compaction. **Object.** Betti and resolution signatures. **Loss.**

$$\mathcal{L}_{\text{dcompact}} = \|b - \text{deq}(Q_b(b))\|^2 + \lambda \text{CE}(\text{quasiiso_class}, \hat{c}). \quad (11.39)$$

Why it helps. It lets the retrieval head compare derived structure without loading full complexes. **Integration.** Store compressed Betti signatures beside vector codes.

N18. Toroidal periodic context cache. **Object.** Phase code $\theta_t \in \mathbb{R}^m / \mathbb{Z}^m$ for recurring tools, periodic reports, or cyclic project states. **Loss.**

$$\mathcal{L}_{\text{torus}} = \|e^{2\pi i \theta_{t+1}} - e^{2\pi i (\theta_t + \omega)}\|^2 + \lambda \text{CE}(\text{periodic_retrieve}, \hat{y}). \quad (11.40)$$

Why it helps. It improves retrieval for periodic memories without storing repeated text. **Integration.** Use for scheduled evaluations, recurring dashboards, and iterative experiments.

N19. Behavior-corridor memory sheaf. **Object.** Tone, personality, rigor, safety, and accuracy sections over memory. **Loss.**

$$\mathcal{L}_{\text{behsheaf}} = \mathcal{L}_{\tilde{C}}(s_{\text{behavior}}) + \lambda \sum_i \text{dist}(s_i, K_i)^2. \quad (11.41)$$

Why it helps. It retrieves not only facts but the appropriate way to use those facts. **Integration.** Store behavior constraints as memory sections; audit style leakage.

N20. Artifact-budget context controller. **Object.** Serialization budget for projections, quantizers, indexes, and certificates. **Controller.**

$$\max_{S \subseteq \mathcal{A}} \Delta \text{bits-per-byte}(S) + \lambda_R \Delta \text{Reason}(S) - \lambda_B \text{bytes}(S) - \lambda_L \text{latency}(S) \quad (11.42)$$

subject to security and prefix-safety constraints. **Why it helps.** It aligns the system with the deployment goal. **Integration.** Solve greedily or by small knapsack over candidate modules after each validation sweep.

12 Worked examples and explicit computations

12.1 Example 1: a minimal MCP trace becomes a retrieval complex

Consider a trace with six events:

$$V = \{u, m, t, c, r, a\}, \quad (12.1)$$

where u is the user request, m is the model planning message, t is a listed tool definition, c is the tool call, r is the tool result, and a is the final answer. The typed edges are

$$u \rightarrow m, \quad m \rightarrow c, \quad t \rightarrow c, \quad c \rightarrow r, \quad r \rightarrow a, \quad (12.2)$$

plus a provenance edge $t \rightarrow r$ asserting that the result belongs to that tool schema. Let the role vector be

$$\deg(u) = (1, 0, 0, 0), \deg(t) = (0, 1, 0, 0), \deg(r) = (0, 0, 1, 0), \deg(a) = (0, 0, 0, 1) \quad (12.3)$$

for query, schema, evidence, and answer. A valid retrieval package for answering from the tool result should contain the 2-simplex $[t, c, r]$ and the path $r \rightarrow a$. If r is retrieved without t , schema validation may fail; if a is generated without r , evidence support fails.

Set a two-coordinate filtration $\alpha = (\rho, \theta)$, where ρ is projected-vector radius and θ is a trust threshold. Suppose t, c, r have pairwise projected distances 0.22, 0.25, 0.31 and trust scores 1.0, 0.95, 0.97, while u and a are outside the retrieval radius. At $\alpha = (0.30, 0.90)$ the complex contains edges $[t, c]$ and $[c, r]$ but not $[t, r]$, so $[t, c, r]$ is absent. At $\alpha = (0.32, 0.90)$ the triangle appears. The lifetime of the 1-cycle $t - c - r - t$ is therefore short; the retrieval package is coherent. If $[t, r]$ never appears, the tool result and tool schema do not glue, and the retriever should either fetch the missing schema or downweight the result.

This tiny computation gives an exact train-time label. The model’s projected retrieval head should learn that t and r need to be retrieved together when the answer depends on schema-valid tool output. The auxiliary improves bits-per-byte only if including the schema/result pair increases the probability of correct answer bytes or reduces later self-correction text. It improves reasoning quality by preventing unsupported tool-result use.

12.2 Example 2: markdown-memory Cech residual

Let a markdown project memory have cover $U_1 = \text{“dataset”}$, $U_2 = \text{“license”}$, and $U_3 = \text{“current experiment”}$. Suppose the local sections are simplified scalar triples

$$s_i = (\text{dataset_id}, \text{license_class}, \text{allowed_use}). \quad (12.4)$$

On overlaps, restrictions keep the common coordinates. Let

$$s_1 = (42, \perp, \perp), \quad (12.5)$$

$$s_2 = (\perp, \text{NC}, \text{research}), \quad (12.6)$$

$$s_3 = (42, \text{Commercial}, \text{product}). \quad (12.7)$$

Then the overlap $U_2 \cap U_3$ has conflict

$$\rho_{2,23}(s_2) - \rho_{3,23}(s_3) = (\text{NC} - \text{Commercial}, \text{research} - \text{product}), \quad (12.8)$$

which is nonzero under any one-hot or learned metric. The Cech residual flags that the current experiment section is inconsistent with the license section. A flat vector retriever may return s_3 because it is recent; the sheafified retriever sees the obstruction and retrieves s_2 as well, or asks for clarification.

The same example yields a local-cohomology alarm. The obstruction is supported on the overlap involving the license block, not on the dataset block. Therefore the answer should not discard the dataset fact; it should modify usage claims. This is why local algebra matters for reasoning quality: it tells the model what to distrust and what remains usable.

12.3 Example 3: path-algebra normal form in a knowledge graph

Let relation types be `paper_has_method`, `method_uses_tool`, and `paper_uses_tool`. A schema rule says

$$\text{method_uses_tool} \circ \text{paper_has_method} = \text{paper_uses_tool}. \quad (12.9)$$

In a path algebra quotient, the two paths have the same normal form. If embeddings use relation operators R_a, R_b, R_c , train

$$\mathcal{L}_{\text{KG}} = \|R_{\text{method_uses_tool}} R_{\text{paper_has_method}} z_p - R_{\text{paper_uses_tool}} z_p\|^2. \quad (12.10)$$

This is also a toric binomial relation: if x_a, x_b, x_c encode relation atoms, then $x_b x_a - x_c$ lies in the ideal of schema-equivalent paths. The retrieval benefit is concrete. A query about tools used by a paper may retrieve either an explicit paper-tool edge or the method-mediated path. The normal-form loss teaches the projection head not to rank one route as unrelated to the other.

12.4 Example 4: a Cox context code with invalid coactivation

Let four one-dimensional cones index variables

$$x_E = \text{evidence}, \quad x_V = \text{verifier}, \quad x_S = \text{safety consent}, \quad x_A = \text{answer commitment}. \quad (12.11)$$

A useful answer package has monomial $x_E x_V x_A$. A sensitive tool package requires x_S . The nonface $\{x_A, x_{\neg E}\}$ says answer commitment without evidence is invalid; the nonface $\{x_{\text{sensitive tool}}, x_{\neg S}\}$ says a sensitive tool without consent is invalid. The Stanley-Reisner penalty is

$$\mathcal{L}_{\text{invalid}} = p_A p_{\neg E} + p_{\text{sensitive}} p_{\neg S}. \quad (12.12)$$

This is cheaper and more compositional than a large safety classifier. It is not a replacement for safety policy, but it gives the model an algebraic memory of forbidden feature combinations.

12.5 Example 5: retrieval projection margin calculation

Suppose the exact reranker gives scores

$$s_{v^*} = 4.10, \quad s_{v_1} = 3.71, \quad s_{v_2} = 3.40. \quad (12.13)$$

The margin is $\gamma = 0.39$. If a projection and quantizer are audited to have score error at most $\varepsilon = 0.12$ on this candidate set, then $2\varepsilon = 0.24 < 0.39$ and the top candidate is stable. If the quantized export error rises to $\varepsilon = 0.22$, the theorem no longer certifies stability. The system may still retrieve correctly, but it should not count this candidate as margin-certified. This gives a practical diagnostic: store not only nearest-neighbor ids, but an error certificate for the projection/quantizer family.

12.6 Example 6: tableau compression of ontology schemas

A biomedical memory graph often has nested fields: disease, molecular pathway, intervention, assay, evidence type, and population. If the schema hierarchy is represented by semistandard tableaux, rows encode compatible refinements and columns encode strict hierarchy. The weight vector counts field usage. A generated context package with the wrong hierarchy creates an inversion. The loss

$$\mathcal{L}_{\text{schema-tab}} = \text{Inv}(T) + \|\text{wt}(T) - \text{wt}(T^*)\|_1 \quad (12.14)$$

encourages canonical schema order. This does not make the mathematics decorative; it improves retrieval packing because repeated nested schemas share compact normal forms.

13 Detailed architecture of the MCP retrieval head

13.1 Five-stage retrieval

The deployed retrieval head has five stages.

1. **Prefix-safe query encoding.** Build a query packet from the current prefix, current MCP state, user request, and allowed memory roots. The packet includes no future validation bytes.
2. **Projected approximate search.** Use $P_q h_q$ against quantized projected vectors $Q(P_z z_v)$ to produce a small candidate set.
3. **Graph expansion.** Add typed neighbors within radius R under permission constraints. Expansion is relation-aware: a tool result expands to its tool schema and provenance; a markdown block expands to parent headings and linked blocks; a KG entity expands by relation types allowed by the query.
4. **Algebraic reranking.** Rerank by sheaf-gluing residual, Cox-degree compatibility, path-algebra normal form, Cech obstruction, and verifier/support fields.
5. **Context packing.** Select a token-budgeted package that maximizes marginal utility and rejects Stanley-Reisner nonfaces.

13.2 Retriever as a morphism of sheafified graphs

Let $\mathbb{D}$ be the full context database and $R_x \subset \mathbb{D}$ the retrieved package for query x . The inclusion $i : R_x \hookrightarrow \mathbb{D}$ should preserve the sheaf sections needed by the answer. The direct-image view says the retrieved package is useful when

$$H^0(R_x, i^* \mathcal{F}) \rightarrow H^0(Q_x, \mathcal{F}_{\text{needed}}) \quad (13.1)$$

contains the needed answer section and has small obstruction in H^1 . In implementation this becomes a reranking feature:

$$S_{\text{sheaf}}(x, R) = \alpha \dim \text{im } H^0(R, i^* \mathcal{F}) - \beta \dim H^1(R, i^* \mathcal{F}) - \gamma \text{Conflict}(R). \quad (13.2)$$

The dimensions are estimated by exact finite linear algebra on small retrieved packages or by trained surrogates.

13.3 Training data construction

A high-quality ToricGT III corpus should include paired queries and context packages. Positive packages come from final answers, citations, successful tool calls, accepted memory updates, or verifier traces. Hard negatives are more important than random negatives. They include:

- semantically similar but stale memory blocks;
- correct entities with wrong relation paths;
- tool results without matching schema;
- resources outside permission boundary;
- markdown blocks that conflict on overlaps;
- style/personality memories from incompatible chambers;
- answer-commitment packages missing evidence or verifier sections.

This negative set is exactly where graph structure helps. A flat embedding often confuses these cases; the algebraic sidecar separates them.

14 Deployment blueprint for an MCP server

14.1 Server interface

A ToricGT III retrieval server can be exposed as an MCP server with tools and resources. The server should expose at least:

- `context.search`: returns candidate records with provenance and scores;
- `context.pack`: returns a budgeted context package;
- `memory.read`: returns authorized markdown sections and sheaf summaries;
- `memory.write_proposal`: proposes edits with mapping-cone and Cech audits;
- `kg.expand`: expands typed knowledge-graph neighborhoods;
- `audit.context`: reports schema, permission, Cech, Cox-degree, and quantization checks.

Each tool should validate inputs, respect access controls, provide structured output, and return actionable tool-execution errors when the request is semantically invalid but structurally well formed.

14.2 Structured outputs

A context search result should not be just text. It should include a typed payload:

Listing 2: Structured retrieval result sketch.

```
class RetrievalResult(BaseModel):
    record_ids: list[str]
    projected_scores: list[float]
    rerank_scores: list[float]
    graph_edges: list[tuple[str, str, str]]
    cox_degrees: list[list[int]]
    sheaf_residual: float
    schema_valid: bool
    permission_valid: bool
    provenance_summary: dict
    quantization_error_bound: float
    recommended_use: Literal["cite", "background", "verify", "do_not_use"]
```

Structured outputs let the model reason about why context was retrieved and how safely it can be used. They also support post-hoc audit: if a final answer used a record marked `do_not_use`, the failure is localizable.

14.3 Memory write protocol

Memory writes are riskier than reads. A proposed write should be treated as a morphism

$$W : \mathbb{D}_{\text{old}} \rightarrow \mathbb{D}_{\text{new}}. \quad (14.1)$$

Before committing, compute four checks: schema validity, permission validity, Cech compatibility with adjacent markdown sections, and mapping-cone residual against the derived context object. The write is safe when it either preserves structure or declares a localized intended change. A large unexplained cone homology is an edit alarm.

15 Additional theorem: context utility under perfect retrieval

Theorem 15.1 (Utility decomposition under exact context selection). *Let C be a finite set of candidate context records and let $S \subset C$ be a selected package. Suppose the next-byte distribution factors through a sufficient context statistic $T(S)$, so that*

$$B_t \perp S \mid (\mathcal{F}_t, T(S)). \quad (15.1)$$

Then the optimal expected log-loss improvement from selecting S equals

$$I(B_t; T(S) \mid \mathcal{F}_t). \quad (15.2)$$

If T is additive over independent sheaf components, this information decomposes as a sum over connected components minus overlap corrections measured by Cech terms.

Proof. The first statement is the side-information theorem applied to $T(S)$ instead of S . The conditional independence assumption says S contains no additional predictive information after $T(S)$ is known. For the decomposition, use the chain rule for mutual information over components. Overlap variables are counted twice unless corrected; in the sheaf setting, those overlaps are exactly the restrictions appearing in the Cech complex, so the correction terms are functions of the overlap sections. $\square$

16 Risk controls and failure modes

16.1 False positives

There are four recurring false positives.

1. **Semantic neighbor, wrong permission.** The vector is close but the record is not allowed for the current principal.
2. **Current-looking stale memory.** A recent summary copied an old fact; metadata freshness is misleading unless the section content and provenance agree.
3. **Tool result without schema.** A tool returned text that looks relevant but fails output-schema or intent-cycle checks.
4. **Behavior leakage.** A prior memory contains a style or persona state that is inappropriate for the current context.

The algebraic controls map directly to these failures: permission masks, Cech gluing, schema preservation, and behavior-corridor sheaves.

16.2 Ablation ladder

The minimum ablation ladder is:

1. no retrieval;
2. flat vector retrieval;
3. projected graph retrieval;
4. projected graph retrieval plus quantization;
5. graph retrieval plus sheaf/schema reranking;
6. full teacher-side toric/derived audits;
7. quantized student artifact.

A system is credited only if it beats its immediate predecessor on matched seeds. Report global held-out bits-per-byte, slice bits-per-byte, answer accuracy, retrieval recall, schema validity, Cech residual, permission violations, style-corridor violations, latency, and artifact bytes.

17 Implementation methodology

17.1 Offline build

Listing 3: Offline index build sketch.

```
for event in mcp_trace_log:
    g.add_event(event)
for md_file in markdown_memory_files:
    blocks = parse_markdown_blocks(md_file)
    g.add_block_graph(blocks)
for kg_shard in knowledge_graph_shards:
    g.add_typed_edges(kg_shard)

sections = build_memory_sheaves(g)
complexes = build_filtered_complexes(g, thresholds)
certs = exact_sidecar_audits(g, sections, complexes)
Z = encode_nodes_edges(g, text_encoder, graph_encoder)
P = train_or_load_projection(Z, hard_pairs)
Q = fit_quantizer(P @ Z, cox_degree_constraints=True)
ann = build_ann_index(Q(P @ Z), adjacency=g.adjacency)
serialize(graph=g.slim(), projected=P @ Z, codes=Q, cert_summaries=compress(certs))
```

The index is built in layers. The raw store remains outside the compact artifact when possible. The deployable artifact stores projected vectors, quantized codes, lightweight adjacency, and summaries necessary for prefix-safe retrieval. Exact certificates are kept in the teacher or audit service unless they prove deployment utility.

17.2 Online retrieval

Listing 4: Online retrieval sketch.

```
def retrieve_context(prefix, user_query, mcp_state):
    x = make_query_packet(prefix, user_query, mcp_state)
    h = query_encoder(x)
    h_small = P_q @ h
    candidates = ann.topk(h_small, k=k0)
    expanded = graph_expand(candidates, radius=R, permissions=x.permissions)
    features = build_rerank_features(expanded, x)
    scores = reranker(h, features)
    package = context_packer(scores, budget=tokens, nonfaces=SR_rules)
    audit = cheap_prefix_audit(package, x)
    if audit.fail:
        package = fallback_package(x)
    return package
```

17.3 Training schedule

Use a staged schedule.

1. Train baseline language/reasoning model with ordinary retrieval disabled or frozen.
2. Build audit-only graph/memory/MCP features and estimate conditional information.
3. Train projection and quantization heads with encoder frozen.
4. Enable retrieval loss and graph expansion, then Cech/sheaf and schema losses.
5. Add derived, toric, tableau, and ergodic objectives in teacher mode.
6. Distill only useful codes into the student; run quantized round-trip bits-per-byte, retrieval, verifier, safety, and latency ablations.

18 Evaluation protocol

Evaluation should report global and slice metrics. The essential slices are: ordinary natural text; MCP-heavy tool use; markdown memory editing; knowledge-graph question answering; multi-hop evidence retrieval; long-context project memory; safety/permission prompts; style/tone/personality consistency; and algebraic/combinatorial synthetic tasks.

Metric	Meaning
Held-out bits-per-byte	Primary compression score under strict prefix-safe retrieval.
Retrieval Recall@k	Fraction of examples with a useful record in projected or graph-expanded candidates.
Context Utility@budget	Verified answer improvement per token or artifact byte.
Schema validity	Fraction of tool calls and resource parses satisfying declared schema.
Cech residual	Local-to-global memory inconsistency.
Derived residual	Chain-map or mapping-cone failure under memory updates.
Quantized score delta	Difference between full and quantized retrieval/reranking scores.
Behavior corridor violation	Tone/personality/safety/accuracy section outside declared corridor.
Latency and artifact bytes	Deployment cost of vectors, adjacency, quantizers, and certificates.
Exact/surrogate agreement	Correlation and calibration between GPU losses and exact finite audits.

19 Discussion and limitations

ToricGT III is intentionally conservative about deployment claims. The theorems prove finite generation, stability under margins, gluing under zero residual, quantization error bounds, and ergodic interpretations under stated assumptions. They do not prove that a particular implementation improves held-out bits-per-byte. That must be established by ablations under strict prefix-safe scoring.

The most likely immediately useful components are simple: graph-structured vector retrieval, learned projections, quantization-aware retrieval training, schema preservation, Cech memory gluing, Stanley-Reisner nonfaces, Cox role degrees, and artifact-budget gates. The more elaborate objects - derived categories, spectral alarms, tableaux, root-system combinatorics, toroidal recurrence, and local cohomology - belong first in teacher-side audits, synthetic curricula, and rerankers. They should enter the serialized MCP retrieval artifact only after proving net utility.

The paper also separates security from retrieval quality. A retrieved context may be relevant but not permitted. A tool may be useful but unsafe without confirmation. A model may be accurate but tonally or behaviorally inconsistent with the declared corridor. ToricGT III therefore stores permissions, provenance, and behavior sections as first-class graph/sheaf data, not as comments inside raw text.

20 Conclusion

ToricGT III extends the ToricGT program from internal reasoning geometry to the external context substrate of agentic systems. MCP traces, markdown memories, and knowledge graphs are not flat bags of text. They are typed, dynamic, sheafified, graph-structured algebraic objects. Vector databases for reasoning agents should preserve that structure: vectors need adjacency, filtrations, quantized toric codes, sheaf restrictions, provenance, and exact finite audits. The learned projection head makes retrieval efficient; graph expansion and reranking recover structure; toric quantization makes the artifact compact; derived and Cech residuals make updates and memory gluing auditable; ergodic diagnostics control long-run behavior. The deployment rule remains unchanged from ToricGT I and II: retain only those pieces that improve held-out bits-per-byte, verified reasoning, retrieval quality, behavior control, or safety under realistic artifact and latency budgets.

A Additional proof details

A.1 Submodular context packing

If utility $U(S)$ is monotone submodular and cost is cardinality with budget k , the greedy algorithm attains a $(1 - 1/e)$ approximation. In ToricGT III, learned utility is not guaranteed submodular, but the decomposition

$$U(S) = \sum_{v \in S} a_v - \sum_{u, v \in S} r_{uv} + \sum_{\sigma \subset S} b_\sigma \quad (\text{A.1})$$

can be regularized toward diminishing returns by penalizing positive second differences outside allowed higher-order simplices.

A.2 Graph expansion and recall

Let $P(x)$ be the set of useful records and $C_k(x)$ the ANN candidate set. If every useful record is within graph distance R of at least one approximate candidate with probability p , then graph-expanded recall is at least p . Training should therefore mine not only positive records, but positive neighborhoods.

B Normalized record schema

Listing 5: Normalized ToricGT III context record.

```
@dataclass
class ContextRecord:
    id: str
    kind: Literal["message", "tool", "resource", "prompt", "markdown", "kg", "verifier"]
    text: str
    vector_full: FloatTensor # high-dimensional, teacher/store side
    vector_proj: FloatTensor # deployable retrieval vector
    quant_code: LongTensor # compact code
    node_type: int
    cox_degree: LongTensor
    one_dimensional_cone_ids: LongTensor
    adjacency: list[tuple[str, str]]
    schema_hash: Optional[str]
    permissions: PermissionMask
    provenance: Provenance
    freshness: float
```

```

sheaf_section_id: Optional[str]
certificate_summary: Optional[SmallTensor]
context_span: tuple[int, int] # token interval in the long context
block_id: int # tropical ring attention block id
marginal_bits_per_byte_utility: float
cech_residual: float
behavior_corridor_residual: float

```

C Bibliography

References

- [1] Model Context Protocol specification, version 2025-06-18. <https://modelcontextprotocol.io/specification/2025-06-18>. Accessed 2026-06-08.
- [2] Model Context Protocol server tools specification, version 2025-06-18. <https://modelcontextprotocol.io/specification/2025-06-18/server/tools>. Accessed 2026-06-08.
- [3] Model Context Protocol server resources specification, version 2025-06-18. <https://modelcontextprotocol.io/specification/2025-06-18/server/resources>. Accessed 2026-06-08.
- [4] Model Context Protocol client roots specification, version 2025-06-18. <https://modelcontextprotocol.io/specification/2025-06-18/client/roots>. Accessed 2026-06-08.
- [5] David A. Cox, John B. Little, and Henry K. Schenck. *Toric Varieties*. American Mathematical Society, 2011.
- [6] Amelie Schreiber. *ToricGT with Toric BGG Supervision: Graph-Token Reasoning, Hypertoric Category O, and bits-per-byte-Constrained Training*. 2026.
- [7] Amelie Schreiber. *ToricGT II: Dynamical, Ergodic, and Derived Persistence Geometry for Iterative Reasoning Agents*. 2026.
- [8] Herbert Edelsbrunner, David Letscher, and Afra Zomorodian. Topological persistence and simplification. *Discrete and Computational Geometry*, 2002.
- [9] Afra Zomorodian and Gunnar Carlsson. Computing persistent homology. *Discrete and Computational Geometry*, 2005.
- [10] Frederic Chazal, Vin de Silva, Marc Glisse, and Steve Oudot. *The Structure and Stability of Persistence Modules*. Springer, 2016.
- [11] Yu. A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE TPAMI*, 2020.
- [12] Jeff Johnson, Matthijs Douze, and Herve Jegou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 2019.
- [13] Herve Jegou, Matthijs Douze, and Cordelia Schmid. Product quantization for nearest neighbor search. *IEEE TPAMI*, 2011.

- [14] Vladimir Karpukhin et al. Dense passage retrieval for open-domain question answering. *EMNLP*, 2020.
- [15] Patrick Lewis et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. *NeurIPS*, 2020.
- [16] Antoine Bordes et al. Translating embeddings for modeling multi-relational data. *NeurIPS*, 2013.
- [17] Michael Schlichtkrull et al. Modeling relational data with graph convolutional networks. *ESWC*, 2018.
- [18] George D. Birkhoff. Proof of the ergodic theorem. *Proceedings of the National Academy of Sciences*, 1931.
- [19] William B. Johnson and Joram Lindenstrauss. Extensions of Lipschitz mappings into a Hilbert space. *Contemporary Mathematics*, 1984.
- [20] Justin Curry. *Sheaves, Cosheaves and Applications*. PhD thesis, University of Pennsylvania, 2014.
- [21] Robert Ghrist. Elementary applied topology. Createspace, 2014.